

湖南科技大学

Hunan University of Science and Technology

《操作系统课程设计》指导书

2025-2026 第二学期

指导单位：湖南科技大学 计算机科学与工程学院

日 期：第 17-18 周（2026.6.22-2026.7.5）

2026 年 5 月

一、课程设计目的

操作系统是计算机科学与技术专业学生必须学习和掌握的一门理论性和实践性并重的核心主干课程和专业基础课程，2009 年被列为全国硕士研究生相关专业基础统考科目之一。操作系统具有技术综合性强、设计技巧高的特点，其基本概念、设计思想、算法和技术可运用到软件开发的其它领域。同时，由于操作系统是一种大型的、复杂软件系统，参与操作系统的设计和开发，有助于学生获得从事大型、复杂系统设计、开发和团队合作的经验。无论是在操作系统上进行应用软件开发，还是从事操作系统本身的研究、设计和开发，都需要理解和掌握操作系统。理解和掌握操作系统的最好途径，就是实践。因此实验和设计实践也是操作系统课程中的重要环节。

教育部高等学校计算机科学与技术教学指导委员会 2009 年编制了《高等学校计算机科学与技术专业核心课程教学实施方案》，其中把操作系统实验和实践项目分为使用管理、观察分析、编程修改和系统设计与实现 4 个层次。

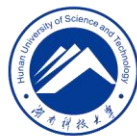
为了体现课程的内容要求和能力培养要求，体现课程特色，强化操作系统研究与开发的实际动手能力，本指导书中综合设计了多层次、多单元的操作系统课程实验和课程设计方案，从实验类型维度分为 4 个实践层次，即使用与管理、观察与分析、编程与修改、设计与实现。涉及操作系统的进程管理、存储管理、设备管理、文件管理和用户接口等多个方面，涵盖大纲规定的认识性、验证性、研究性、综合性等实验。在难度维度上分为用户级和内核初级、内核高级三种难度。针对 Linux 和 Windows 操作系统设计了有特色的实验单元。这 4 个实践层次、三种难度的实验单元相对独立，又环环相扣，逐步深入，前一步实践的结果为后一步实践打下基础。

本课程设计的学习目标如下：

学习目标 1：具备将操作系统的基本概念和基础理论运用于实际工程的能力。

学习目标 2：掌握基本的分析方法、设计方法、调试方法和优化方法，具有构建小型、中型软件系统的能力。

学习目标 3：熟练掌握操作系统领域的相关算法，可运用高级语言实现操作系统的各个组成模块。

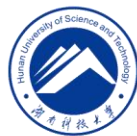


本课程在 OBE 教学理念指导下对毕业要求指标点的支撑关系如下图所示：

专业毕业要求	专业毕业要求指标点	对应的学习目标
A.工程知识：能够将数学、自然科学、计算、工程基础和专业知用于解决复杂工程问题。	A3 掌握软硬件理论专业知识，用于物联网领域的软硬件系统及复杂工程问题的需求分析、设计、开发与实现。	学习目标 1
C.设计/开发解决方案：能够针对复杂工程问题设计和开发解决方案，设计满足特定需求的系统、单元（部件）或工艺流程，体现创新性，并从健康、安全与环境、全生命周期成本与净零碳要求、法律与伦理、社会与文化等角度考虑可行性。	C1 能够基于物联网工程问题的需求分析，确定其设计目标及具体方案。	学习目标 2
E.使用现代工具：能够针对复杂工程问题，开发、选择与使用恰当的技术、资源、现代工程工具和信息技术工具，包括对复杂工程问题的预测与模拟，并能够理解其局限性。	E2 能够基于集成开发工具与环境、开源资源以及第三方平台，开发物联网工程相关软硬件系统，实现对物联网工程问题的预测与模拟，并能够理解其局限性。	学习目标 3

二、课程设计纪律要求

1. 请每位同学在课程设计之前做好充分的准备工作，按指导老师要求下载相关软件，事先准备并熟悉实验的软硬件环境，预习各项实验任务。
2. 在课程设计期间，应按时上下课，不迟到早退，请假需出示有效证明。
3. 在课程设计期间，应遵守实验室的规章制度，爱护实验设备，不得在实验室做与实验无关的事情，禁止利用实验室条件进行看电影、打游戏、聊天等与老师布置的上机任务不相干的事情，影响其他同学的上机。对于实验设备出现的问题，要及时向指导老师汇报。
4. 在课程设计期间，应每天按时填写课程设计工作日志，并接受指导老师的定期检查。没有日志的学生课程设计不及格。
5. 在课程设计期间，应独立完成程序设计、撰写课程设计报告、不允许相互抄袭，否则不及格。
6. 在课程设计过程中，要求理论联系实际，注意所学知识的综合运用，注重创造力的培养。
7. 课程设计过程中，要始终坚持严肃认真、实事求是、独立思考、虚心学习、刻苦钻研的态度，要加强对基础理论、基本知识与基本技能的掌握和运用。
8. 为了避免选题集中在少数几个实验，指导老师可以在第一天开展多样化选题工作，由学习委员负责组织，在设计开始第一天将每位同学的选题结果汇报给指导老师。
9. 本次课程设计建议由学生独立完成，不进行分组。
10. 时间安排为 2 周，建议前三天完成验证性实验，余下的时间用于设计类实验。指导老师可以此作为参考进行阶段性测试。



三、考核及报告要求

1. 课程设计结束后,作为归档材料,每位同学需提交 2 份纸质文件——课程设计报告和日志文件。前者格式要求参见附件 1,封面单面打印,封面之外正文部分内容建议使用双面打印,每个实验的报告篇幅建议大约 3-4 面,课程设计报告的总面数大约 30 面左右为宜。后者格式要求参见附件 2。
2. 最终的课程设计报告要求格式规范,语言通顺。在“实验内容”部分写明使用的数据结构,画出算法的程序流程图、关键的源程序代码,非必要不得直接复制本指导书的内容,违规者不及格。在“实验结果与分析”部分,可以用图片、表格、文字等形式进行记录和分析,不能仅罗列现象不作任何分析。在“小结与心得体会”部分,要写出个人的真情实感、切身体会,不允许废话敷衍写成流水账。
3. 课程成绩包括 4 个部分,分别为上机实验、设计报告、课堂表现、创新得分,分别反映学生的方案设计与分析能力、课程设计报告质量、工作态度、综合性与创新性。指导教师根据实践教学期间学生的态度、能力和成效,综合评定成绩。具体如下:上机实验由指导教师根据现场上机测试给予成绩,设计报告由指导教师参考报告评分标准给予成绩,课堂表现由指导教师根据学生考勤、提问、师生交流情况给予成绩,创新得分由指导教师根据学生在课堂、作业、报告中体现的创新性给予成绩。其中**方案设计与分析能力占总成绩的 30%,课程设计报告质量占总成绩的 40%,工作态度占总成绩的 20%,综合性与创新性占总成绩的 10%。**按“**≥90 分为优秀、80 分-89 分为良好、70 分-79 分为中等、60-69 分为及格、60 分以下为不及格**”的标准转换为等级制,录入教务系统,记入学生成绩档案。
4. 报告评分标准方面,本指导书提供了 5 个 A 类实验、9 个 B 类实验、7 个 C 类实验。
 - a) A 类为认识性、验证性实验,难度为用户层初级,本书已提供全部代码,要求每位同学编译运行代码,观察记录运行结果,回答相关问题。
 - b) B 类为综合性实验,难度为用户层高级和内核层初级,本书仅给出提示和要求,同学需要自主编程实现理论教学中涉及的重要算法或子系统。
 - c) C 类为研究性实验,难度为内核层高级,供学有余力的同学选做。C 类实验要求学生从零开始自行搭建一个操作系统可视化平台 OS256,其中包含一个硬件模拟器 PC256、一个汇编器 assembler256,并在其中开展操作系统相关实验,最终在 OS256 中实现一个微型操作系统。C 类实验符合教育部对于提高课程的“高阶性、创新性和挑战度”的要求。
 - d) **所有同学必须完成全部 5 个 A 类实验、至少 5 个 B 类实验,即 5+5 为及格线。其它成绩等次由指导老师根据实际情况决定。建议 5+6 为中等,5+7 为良好,5+8 以上为优秀。C 类实验的成绩评定由指导老师决定。**

四、成绩评定细则

1. 优秀:上机实验答辩思路清晰,完全正确地回答指导教师的所有提问。设计报告格式美观、内容详实、算法正确、心得体会深刻。课堂表现突出,考勤满分,积极与指导教师交流。创新得分方面,学生在课堂、作业、报告中表现出显著的创新意识和创新能力。
2. 良好:上机实验答辩思路清晰,正确地回答指导教师的大部分提问。设计报告格式较美

观、内容较详实、算法正确、心得体会较深刻。课堂表现良好，考勤满分，积极与指导教师交流。创新得分方面，学生在课堂、作业、报告中表现出较强的创新意识和创新能力。

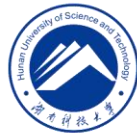
3. 中等：上机实验答辩思路较清晰，正确地回答指导教师的多数提问。设计报告格式正确、内容达到基本要求、算法基本正确、心得体会较诚恳。课堂表现一般，考勤满分。创新得分方面，学生在课堂、作业、报告中表现出一定的创新意识和创新能力。
4. 及格：上机实验答辩思路不太清晰，只能正确地回答指导教师的少量提问。设计报告格式一般、内容不丰富、算法存在瑕疵、心得体会敷衍。课堂表现一般，考勤未满分。创新得分方面，学生在课堂、作业、报告中未表现出明确的创新意识和创新能力。
5. 不及格：上机实验答辩思路不清晰，不能正确回答任何问题。设计报告存在格式问题、内容少、算法错、心得体会敷衍。课堂表现不佳，考勤失分较多，排斥与指导教师交流。创新得分方面，学生在课堂、作业、报告中未表现出任何创新意识和创新能力。

五、关于 Linux 实验环境的选择和搭建

1. 湖南科技大学实验室的机器上默认安装了 Windows+Linux 双操作系统。可以在开机时选择进入 Linux。
2. 在自己的机器的 windows 中安装虚拟机，在虚拟机中安装 Linux。
3. 在自己的机器的 windows 中使用 WSL2。
4. 远程租用云电脑。
5. 打开蓝桥云课网站 <https://www.lanqiao.cn/courses/>，在手机注册登录后（免费），选择课程《Linux 基础入门》，即可利用其 Linux 虚拟仿真平台，完成本课程设计中的 Linux 相关的所有实验。

六、参考资料

- [1] 任爱华等. 操作系统实用教程（第三版）实验指导[M]. 北京：清华大学出版社，2024.
- [2] 王红玲等. 计算机操作系统实验指导[M]. 北京：人民邮电出版社，2023.
- [3] 袁宝华等. 操作系统实验教程[M]. 北京：清华大学出版社，2023.
- [4] 陆静. Linux 操作系统实验指导[M]. 北京：电子工业出版社，2020.
- [5] 罗宇、陈燕晖等. Linux 操作系统实验教程[M]. 北京：电子工业出版社，2009.



目 录

题目 A1: Windows 进程管理.....	7
题目 A2: Linux 进程控制	15
题目 A3: Linux 进程间通信	18
题目 A4: Windows 线程的互斥与同步.....	24
题目 A5: Windows 内存管理.....	29
题目 B1: 模拟实现银行家算法	37
题目 B2: 模拟实现页面置换算法	39
题目 B3: 模拟实现磁盘调度算法	40
题目 B4: 使用信号量解决读者写者问题	41
题目 B5: 使用信号量解决哲学家进餐问题	42
题目 B6: 模拟实现命令行解释器	43
题目 B7: 模拟实现进程调度	44
题目 B8: 模拟实现动态分区	46
题目 B9: 模拟实现简单文件系统	48
题目 C1: 熟悉 OS256.....	57
题目 C2: 单道批处理	59
题目 C3: 多道批处理与进程	61
题目 C4: 互斥同步	63
题目 C5: 死锁.....	64
题目 C6: 调度.....	66
题目 C7: 分段分页	68
附录: Linux 编程基础	69

A 组（基础题）

题目 A1: Windows 进程管理

特别说明：Windows 下的实验默认以 VC++ 6.0 为 IDE（Integrated Development Environment，集成开发环境），如果该 IDE 在机房机器上出现问题，也可以替换为 codeblock 等其它 IDE，实验内容不变。

1、实验目的

- （1）学会使用 VC 编写基本的 Win32 Consol Application（控制台应用程序）。
- （2）通过创建进程、观察正在运行的进程和终止进程的程序设计和调试操作，进一步熟悉操作系统的进程概念，理解 Windows 进程的“一生”。
- （3）通过阅读和分析实验程序，学习创建进程、观察进程、终止进程以及父子进程同步的基本程序设计方法。

2、背景知识

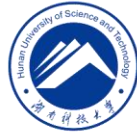
Windows 所创建的每个进程都从调用 CreateProcess() API 函数开始，该函数的任务是在对象管理器子系统内初始化进程对象。每一进程都以调用 ExitProcess() 或 TerminateProcess() API 函数终止。通常应用程序的框架负责调用 ExitProcess() 函数。对于 C++ 运行库来说，这一调用发生在应用程序的 main() 函数返回之后。

（1）创建进程

CreateProcess() 调用的核心参数是可执行文件运行时的文件名及其命令行。表 1-1 详细地列出了每个参数的类型和名称。

表 1-1 CreateProcess() 函数的参数

参数名称	使用目的
LPCTSTR lpApplicationName	全部或部分地指明包括可执行代码的 EXE 文件的文件名
LPCTSTR lpCommandLine	向可执行文件发送的参数
LPSECURITY_ATTRIBUTES lpProcessAttributes	返回进程句柄的安全属性。主要指明这一句柄是否应该由其他子进程所继承
LPSECURITY_ATTRIBUTES lpThreadAttributes	返回进程的主线程的句柄的安全属性
BOOL bInheritHandle	一种标志，告诉系统允许新进程继承创建者进程的句柄
DWORD dwCreationFlage	特殊的创建标志（如 CREATE_SUSPENDED）的位标记
LPVOID lpEnvironment	向新进程发送的一套环境变量；如为 null 值则发送调用者环境
LPCTSTR lpCurrentDirectory	新进程的启动目录
STARTUPINFO lpStartupInfo	STARTUPINFO 结构，包括新进程的输入和输出配置的详情
LPPROCESS_INFORMATION lpProcessInformation	调用的结果块；发送新应用程序的进程和主线程的句柄和 ID



可以指定第一个参数，即应用程序的名称，其中包括相对于当前进程的当前目录的全路径或者利用搜索方法找到的路径；lpCommandLine 参数允许调用者向新应用程序发送数据；接下来的三个参数与进程和它的主线程以及返回的指向该对象的句柄的安全性有关。

然后是标志参数，用以在 dwCreationFlags 参数中指明系统应该给予新进程什么行为。经常使用的标志是 CREATE_SUSPENDED，告诉主线程立刻暂停。当准备好时，应该使用 ResumeThread() API 来启动进程。另一个常用的标志是 CREATE_NEW_CONSOLE，告诉新进程启动自己的控制台窗口，而不是利用父窗口。这一参数还允许设置进程的优先级，用以向系统指明，相对于系统中所有其他的活动进程来说，给此进程多少 CPU 时间。

接着是 CreateProcess() 函数调用所需要的三个通常使用缺省值的参数。第一个参数是 lpEnvironment 参数，指明为新进程提供的环境；第二个参数是 lpCurrentDirectory，可用于向主创进程发送与缺省目录不同的新进程使用的特殊的当前目录；第三个参数是 STARTUPINFO 数据结构所必需的，用于在必要时指明新应用程序的主窗口的外观。

CreateProcess() 的最后一个参数是用于新进程对象及其主线程的句柄和 ID 的返回值缓冲区。以 PROCESS_INFORMATION 结构中返回的句柄调用 CloseHandle() API 函数是重要的，因为如果不将这些句柄关闭的话，有可能危及主创进程终止之前的任何未释放的资源。

(2) 正在运行的进程

如果一个进程拥有至少一个执行线程，则为正在系统中运行的进程。通常，这种进程使用主线程来指示它的存在。当主线程结束时，调用 ExitProcess() API 函数，通知系统终止它所拥有的所有正在运行、准备运行或正在挂起的其他线程。当进程正在运行时，可以查看它的许多特性，其中少数特性也允许加以修改。

首先可查看的进程特性是系统进程标识符 (PID)，可利用 GetCurrentProcessId() API 函数来查看，与 GetCurrentProcess() 相似，对该函数的调用不能失败，但返回的 PID 在整个系统中都可使用。其他的可显示当前进程信息的 API 函数还有 GetStartupInfo() 和 GetProcessShutdownParameters()，可给出进程存活期内的配置详情。

通常，一个进程需要它的运行期环境的信息。例如 API 函数 GetModuleFileName() 和 GetCommandLine()，可以给出用在 CreateProcess() 中的参数以启动应用程序。在创建应用程序时可使用的另一个 API 函数是 IsDebuggerPresent()。

可利用 API 函数 GetGuiResources() 来查看进程的 GUI 资源。此函数既可返回指定进程中的打开的 GUI 对象的数目，也可返回指定进程中打开的 USER 对象的数目。进程的其他性能信息可通过 GetProcessIoCounters()、GetProcessPriorityBoost()、GetProcessTimes() 和 GetProcessWorkingSetSize() API 得到。以上这几个 API 函数都只需要具有 PROCESS_QUERY_INFORMATION 访问权限的指向所感兴趣进程的句柄。

另一个可用于进程信息查询的 API 函数是 GetProcessVersion()。此函数只需感兴趣进程的 PID (进程标识号)。这一 API 函数与 GetVersionEx() 的共同作用，可确定运行进程的系统的版本号。

(3) 终止进程

所有进程都是以调用 ExitProcess() 或者 TerminateProcess() 函数结束的。但最好使用前者而不要使用后者，因为进程是在完成了它的所有关闭“职责”之后以正常的终止方式来调用前者的。而外部进程通常调用后者即突然终止进程的进程，由于关闭时的途径不太正常，有可能引起错误的行为。

TerminateProcess() API 函数只要打开带有 PROCESS_TERMINATE 访问权的进程对象，就可以终止进程，并向系统返回指定的代码。这是一种“野蛮”的终止进程的方式，但是有时却是需要的。

(4) 进程同步

Windows 提供了多种同步对象（synchronization objects），如事件对象（event）、互斥对象（mutex）、信号量对象（semaphore）等，来实现进程之间的同步。本实验只涉及 mutex 和 semaphore。

相关知识，请参考：

mutex 的定义：<https://learn.microsoft.com/zh-cn/windows/win32/sync/mutex-objects>。

mutex 的使用：<https://learn.microsoft.com/zh-cn/windows/win32/sync/using-mutex-objects>。

semaphore 的定义：<https://learn.microsoft.com/zh-cn/windows/win32/sync/semaphore-objects>。

semaphore 的使用：<https://learn.microsoft.com/zh-cn/windows/win32/sync/using-semaphore-objects>。

事件（Event）：用于不同线程间的信号通知，包括单次通知事件和重复通知事件两种类型。事件对象可以通过 CreateEvent 函数创建，使用 SetEvent 函数将其设置为有信号状态，使用 ResetEvent 函数将其设置为无信号状态。WaitForSingleObject 或 WaitForMultipleObjects 函数可以等待事件对象的状态变化。

互斥量（Mutex）：用于控制对共享资源的访问，具有独占性，防止线程之间对共享资源的非法访问。互斥量确保同一时间只有一个线程可以访问共享资源。利用 CreateMutex() API 可创建互斥体，创建时还可以指定一个初始的拥有权标志，通过使用这个标志，只有当线程完成了资源的所有的初始化工作时，才允许创建线程释放互斥体。为了获得互斥体，首先，程可使用 OpenMutex() API 来获得指向对象的句柄；然后，线程将这个句柄提供给一个等待函数。当内核将互斥体对象发送给等待线程时，就表明该线程获得了互斥体的拥有权。当线程获得拥有权时，线程控制了对共享资源的访问。最后，放弃共享资源时需要在该对象上调用 ReleaseMute() API。然后系统负责将互斥体拥有权传递给下一个等待着的线程（由到达时间决定顺序）。

信号量（Semaphore）：基于计数器机制控制并发资源的访问数量。信号量可以允许多个线程同时访问共享资源，但总数有限制。利用 CreateSemaphore() API 来创建信号量对象，WaitForSingleObject()用于等待信号量对象变为可用（即信号量的计数大于 0），该函数就是信号量的 P 操作。ReleaseSemaphore()用于释放信号量对象，增加信号量的计数，相当于信号量的 V 操作。

3、实验内容和步骤

（1）编写基本的 Win32 Consol Application

步骤 1：登录进入 Windows 系统，启动 VC++ 6.0。

步骤 2：在“FILE”菜单中单击“NEW”子菜单，在“projects”选项卡中选择“**Win32 Console Application**”，然后在“Project name”处输入工程名，比如“my”，在“Location”处输入工程目录，在出现的窗口中选择“一个空工程”，点击“完成”，点击“确定”，至此已创建一个新的“Win32控制台应用程序”工程。

步骤 3：在“FILE”菜单中单击“NEW”子菜单，在“Files”选项卡中选择“**C++ Source File**”，然后在“File”处输入 C/C++源程序的文件名，比如“my”，点击“确定”，就为 my 工程新增了一个文件 my.cpp。在 VC++ 6.0 界面左侧的 FileView，以文件视图来观察项目构成，其中 Source Files 下就会出现刚刚新建的 my.cpp。

步骤 4：将 1-1 所示的程序清单复制到新创建的 cpp 文件中。编译成可执行文件。

请注意，复制时请从《操作系统课程设计的验证性实验源程序.c》中复制，不要从 PDF 中复制，以免出现 OCR 识别错误，导致源码编译失败，以下所有实验同。

步骤 5：点击“开始”菜单或按“Win 键”，在键盘输入“cmd”并回车，进入 Windows “命令提示符”窗口。在资源管理器中切换到工程所在目录，用鼠标将 debug 子目录下的可执行文件，如“my.exe”，拖放到“命令提示符”窗口，按回车，观察执行结果。

用这种方式执行时，可以直接在命令行追加参数，如“my x 6”，包含 x 和 6 两个参数。对于 C 语言而言，此时 argc=3，argv[0]指向字符串“my”，argv[1]指向字符串“x”，argv[2]

指向字符串“6”。

如何打开一个已有的工程：文件-打开工作区-找到工程对应的 dsw 文件，点击打开。由于兼容性问题，用 VC++ 6.0 打开一个工程时可能崩溃，解决方法是在工程目录中找到以 dsw 为后缀的文件，如“my.dsw”，将其用鼠标拖入 VC++ 6.0，即可打开对应的工程。

当出现以下提示时，删除 debug 子目录下的可执行文件，如“my.exe”，即可解决。

Linking...

LINK : fatal error LNK1168: cannot open Debug/my.exe for writing

(2) 创建进程

本实验显示了创建子进程的基本框架。该程序只是再一次地启动自身，显示它的 PID 和 CloneID。其中 PID 由操作系统定义，CloneID 是程序内的变量，表示是第几号克隆体。

步骤 1：创建一个“Win32 Consol Application”工程，然后拷贝清单 1-2 中的程序，编译成可执行文件。

步骤 2：在“命令提示符”窗口运行步骤 1 中生成的可执行文件，列出运行结果(一共出现了几个窗口，进程号和 ID 各是多少，可以用截图来列出运行结果)。

步骤 3：在“命令提示符”窗口加入参数“3”重新运行生成的可执行文件，如：

“XX/XX/my.exe 3”

请列出运行结果，并与步骤 2 进行比较，请你解释为什么弹出的窗口数量有所变化。可以将 3 修改为 2、4 等其它数字以验证你的结论。

步骤 4：修改清单 1-2 中的程序，将 nClone 的定义和初始化方法按程序注释中的修改方法进行修改（先尝试第一种修改方法，再尝试第二种），编译成可执行文件，再按步骤 2 中的方式运行，请问两种修改导致执行结果出现什么区别？这种区别是由什么原因导致的？请说明变量 nClone 的作用。请说明:: sscanf(argv[1], "%d", &nClone); 这一句代码的作用。

如果出现永不间断弹出窗口的情况，连续快速按下 ctrl+c。

(3) 父子进程的简单通信及终止进程

步骤 1：创建一个“Win32 Consol Application”工程，然后拷贝清单 1-3 中的程序，编译成可执行文件。

步骤 2：在 VC 的工具栏单击“Execute Program”(执行程序) 按钮，或者按 Ctrl+F5 键，或者在“命令提示符”窗口运行步骤 1 中生成的可执行文件，列出运行结果。

步骤 3：按源程序中注释中的提示（将 child 改为别的字符串），修改源程序 1-3，编译执行，列出运行结果。参考 <https://learn.microsoft.com/en-us/windows/win32/api/> 中关于 CreateProcess() 的描述，说明父子进程是如何进行参数传递的，给出程序 1-3 执行过程的大概描述。

步骤 4：按源程序中注释中的提示（WaitForSingleObject 的参数），修改源程序 1-3，编译执行，列出运行结果。参考 <https://learn.microsoft.com/en-us/windows/win32/api/> 中关于 CreateMutex()、OpenMutex()、ReleaseMutex() 和 WaitFor SingleObject() 的描述，说明父子进程如何利用互斥体进行同步，给出同步过程的一个大概描述。

4、给出实验结论

5、程序清单

清单 1-1 一个简单的 Windows 控制台应用程序

```
// hello 项目
# include <iostream>
void main()
{
```



```
std::cout << "Hello, Win32 Consol Application" << std :: endl ;  
}
```

清单 1-2 创建子进程

```
#include <windows.h>  
#include <iostream>  
#include <stdio.h>  
  
// 创建传递过来的进程的克隆过程并赋予其 ID 值  
void StartClone(int nCloneID)  
{  
    // 提取用于当前可执行文件的文件名  
    TCHAR szFilename[MAX_PATH] ;  
    GetModuleFileName(NULL, szFilename, MAX_PATH) ;  
  
    // 格式化用于子进程的命令行并通知其 EXE 文件名和克隆 ID  
    TCHAR szCmdLine[MAX_PATH];  
    sprintf(szCmdLine, "\"%s\" %d", szFilename, nCloneID);  
  
    // 用于子进程的 STARTUPINFO 结构  
    STARTUPINFO si;  
    ZeroMemory(&si, sizeof(si) ) ;  
    si.cb = sizeof(si) ;           // 必须是本结构的大小  
  
    // 返回的用于子进程的进程信息  
    PROCESS_INFORMATION pi;  
  
    // 利用同样的可执行文件和命令行创建进程，并赋予其子进程的性质  
    BOOL bCreateOK::CreateProcess(  
        szFilename,           // 产生这个 EXE 的应用程序的名称  
        szCmdLine,           // 告诉其行为像一个子进程的标志  
        NULL,                // 缺省的进程安全性  
        NULL,                // 缺省的线程安全性  
        FALSE,               // 不继承句柄  
        CREATE_NEW_CONSOLE,  // 使用新的控制台  
        NULL,                // 新的环境  
        NULL,                // 当前目录  
        &si,                  // 启动信息  
        &pi);                 // 返回的进程信息  
  
    // 对子进程释放引用  
    if (bCreateOK)  
    {  
        CloseHandle(pi.hProcess) ;  
        CloseHandle(pi.hThread) ;  
    }  
}  
  
int main(int argc, char* argv[] )  
{  
    // 确定派生出几个进程，及派生进程在进程列表中的位置  
    int nClone=0;
```



```
//修改语句: int nClone;

//第一次修改: nClone=0;
if (argc > 1)
{
    // 从第二个参数中提取克隆 ID
    :: sscanf(argv[1], "%d", &nClone);
}

//第二次修改: nClone=0;
// 显示进程位置
std :: cout << "Process ID:" << :: GetCurrentProcessId()
              << ", Clone ID:" << nClone
              << std :: endl;
// 检查是否有创建子进程的需要
const int c_nCloneMax=5;
if (nClone < c_nCloneMax)
{
    // 发送新进程的命令行和克隆号
    StartClone(++nClone);
}
// 等待响应键盘输入结束进程
getchar();
return 0;
}
```

清单 1-3 父子进程的简单通信及终止进程的示例程序

```
// procterm 项目
# include <windows.h>
# include <iostream>
# include <stdio.h>
static LPCTSTR g_szMutexName = "w2kdg.ProcTerm.mutex.Suicide";

// 创建当前进程的克隆进程的简单方法
void StartClone()
{
    // 提取当前可执行文件的文件名
    TCHAR szFilename[MAX_PATH];
    GetModuleFileName(NULL, szFilename, MAX_PATH);

    // 格式化用于子进程的命令行, 字符串“child”将作为形参传递给子进程的 main 函数
    TCHAR szCmdLine[MAX_PATH];
    //实验 1-3 步骤 3: 将下句中的字符串 child 改为别的字符串, 重新编译执行
    sprintf(szCmdLine, "\\\"%s\\\" child", szFilename);

    // 子进程的启动信息结构
    STARTUPINFO si;
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si); // 应当是此结构的大小

    // 返回的用于子进程的进程信息
    PROCESS_INFORMATION pi;
```



```
// 用同样的可执行文件名和命令行创建进程，并指明它是一个子进程
BOOL bCreateOK=CreateProcess(
    szFilename,           // 产生的应用程序的名称 (本 EXE 文件)
    szCmdLine,            // 告诉我们这是一个子进程的标志
    NULL,                 // 用于进程的缺省的安全性
    NULL,                 // 用于线程的缺省安全性
    FALSE,                // 不继承句柄
    CREATE_NEW_CONSOLE,   // 创建新窗口
    NULL,                 // 新环境
    NULL,                 // 当前目录
    &si,                  // 启动信息结构
    &pi );                // 返回的进程信息
// 释放指向子进程的引用
if (bCreateOK)
{
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
}

void Parent()
{
    // 创建“自杀”互斥程序体
    HANDLE hMutexSuicide=CreateMutex(
        NULL,              // 缺省的安全性
        TRUE,              // 最初拥有的
        g_szMutexName);    // 互斥体名称
    if (hMutexSuicide != NULL)
    {
        // 创建子进程
        std :: cout << "Creating the child process." << std :: endl;
        StartClone();
        // 指令子进程“杀”掉自身
        std :: cout << "Telling the child process to quit. " << std :: endl;
        //等待父进程的键盘响应
        getch();
        //释放互斥体的所有权，这个信号会发送给子进程的 WaitForSingleObject 过程
        ReleaseMutex(hMutexSuicide);
        // 消除句柄
        CloseHandle(hMutexSuicide);
    }
}

void Child()
{
    // 打开“自杀”互斥体
    HANDLE hMutexSuicide = OpenMutex(
        SYNCHRONIZE,       // 打开用于同步
        FALSE,             // 不需要向下传递
        g_szMutexName);    // 名称
    if (hMutexSuicide != NULL)
    {

```




```
// 报告我们正在等待指令
std :: cout << "Child waiting for suicide instructions. " << std :: endl;

//子进程进入阻塞状态，等待父进程通过互斥体发来的信号
WaitForSingleObject(hMutexSuicide, INFINITE);
//实验 1-3 步骤 4：将上句改为 WaitForSingleObject(hMutexSuicide, 0)，重新编译执行

// 准备好终止，清除句柄
std :: cout << "Child quitting." << std :: endl;
CloseHandle(hMutexSuicide);
}
}

int main(int argc, char* argv[])
{
    // 决定其行为是父进程还是子进程
    if (argc>1 && :: strcmp(argv[1], "child") == 0)
    {
        Child();
    }
    else
    {
        Parent();
    }
    return 0;
}
```

题目 A2: Linux 进程控制

特别说明：本实验需要 LINUX 环境，请按照《附录：Linux 编程基础》创建或获得一个 LINUX 环境。

1、实验目的

通过进程的创建、撤销和运行加深对进程概念和进程并发执行的理解，明确进程和程序之间的区别。

2、背景知识

在 Linux 中创建子进程要使用 `fork()` 函数，执行新的命令要使用 `exec()` 系列函数，等待子进程结束使用 `wait()` 函数，结束终止进程使用 `exit()` 函数。

`fork()` 原型如下：`pid_t fork(void);`

`fork` 建立一个子进程，父进程继续运行，子进程在同样的位置执行同样的程序。对于父进程，`fork()` 返回子进程的 `pid`，对于子进程，`fork()` 返回 0。出错时返回 -1。

`exec` 系列有 6 个函数，原型如下：

`extern char **environ;`

`int execl( const char *path, const char *arg, ...);`

`int execlp( const char *file, const char *arg, ...);`

`int execl( const char *path, const char *arg , ..., char * const envp[]);`

`int execlp( const char *path, char *const argv[]);`

`int execl( const char *filename, char *const argv [], char *const envp[]);`

`int execlp( const char *file, char *const argv[]);`

`exec` 系列函数用新的进程映像替换当前的进程映像。这些函数的第一个参数是待执行程序的路径名(文件名)。这些函数调用成功后不会返回，其进程的正文(text)，数据(data)和栈(stack)段被待执行程序覆盖。但是进程的 PID 和所有打开的文件描述符没有改变，同时悬挂信号被清除，信号重置为缺省行为。

在函数 `execl`、`execlp` 和 `execl` 中，`const char *arg` 以及省略号代表的参数可被视为 `arg0`，`arg1`，...，`argn`。它们合起来描述了指向 NULL 结尾的字符串的指针列表，即执行程序的参数列表。作为约定，第一个 `arg` 参数应该指向执行程序名自身，参数列表必须用 NULL 指针结束。

`execlp` 和 `execlp` 函数提供指向 NULL 结尾的字符串的指针数组作为新程序的参数列表。作为约定，指针数组中第一个元素应该指向执行程序名自身。指针数组必须用 NULL 指针结束。

`execl` 函数同时说明了执行进程的环境(environment)，它在 NULL 指针后面要求一个附加参数，NULL 指针用于结束参数列表，或者说，`argv` 数组。这个附加参数是指向 NULL 结尾的字符串的指针数组，它必须用 NULL 指针结束。其它函数从当前进程的 `environ` 外部变量中获取新进程的环境。

`execlp` 和 `execlp` 可根据 `path` 搜索合适的程序运行，其它则需要给出程序全路径。

`Execve()` 类似 `execlp()`，但是加上了环境的处理。

`Wait()` 和 `waitpid()` 可用来等待子进程结束。函数原型：

`#include <sys/wait.h>`

`pid_t wait(int *stat_loc);`

`pid_t waitpid(pid_t pid, int *stat_loc, int options);`

当进程调用 `wait`，它将进入睡眠状态直到有一个子进程结束。`wait` 函数返回子进程的进程 `id`，`stat_loc` 中返回子进程的退出状态。

`waitpid` 的第一个参数 `pid` 的意义：

`pid > 0`：等待进程 `id` 为 `pid` 的子进程。

`pid == 0`：等待与自己同组的任意子进程。

`pid == -1`：等待任意一个子进程

`pid < -1`：等待进程组号为 `-pid` 的任意子进程。

因此，`wait(&stat)` 等价于 `waitpid(-1, &stat, 0)`，`waitpid` 第三个参数 `option` 可以是 `0`，`WNOHANG`，`WUNTRACED` 或这几者的组合。

3、实验内容和步骤

(1) 任务一：进程的创建

任务要求：编写一段程序，使用系统调用 `fork()` 创建一个子进程。当此程序运行时，在系统中有一个父进程和一个子进程活动。让每一个进程在屏幕上分别显示字符：父进程显示字符 “b”；子进程显示字符 “a”，另外父子进程都显示字符 “c”。

步骤 1：使用 `vi` 或 `gedit` 新建一个 `fork_demo.c` 程序，然后拷贝清单 2-1 中的程序，使用 `cc` 或者 `gcc` 编译成可执行文件 `fork_demo`。例如，可以使用 `gcc -o fork_demo fork_demo.c` 完成编译。

步骤 2：在命令行输入 `./fork_demo` 运行该程序。

步骤 3：多次运行程序，观察屏幕上的显示结果，并分析多次运行为什么会出现不同的结果。

(2) 任务二：子进程执行新任务

任务要求：编写一段程序，使用系统调用 `fork()` 创建一个子进程。子进程通过系统调用 `exec` 更换自己原有的执行代码，转去执行 Linux 命令 `/bin/ls` (显示当前目录的列表)，然后调用 `exit()` 函数结束。父进程则调用 `waitpid()` 等待子进程结束，并在子进程结束后显示子进程的标识符，然后正常结束。程序执行过程如图 2-1 所示。

步骤 1：使用 `vi` 或 `gedit` 新建一个 `exec_demo.c` 程序，然后拷贝清单 2-2 中的程序（该程序的执行如图 2-1 所示），使用 `cc` 或者 `gcc` 编译成可执行文件 `exec_demo`。例如，可以使用 `gcc -o exec_demo exec_demo.c` 完成编译。

步骤 2：在命令行输入 `./exec_demo` 运行该程序。

步骤 3：观察该程序在屏幕上的显示结果，并分析。

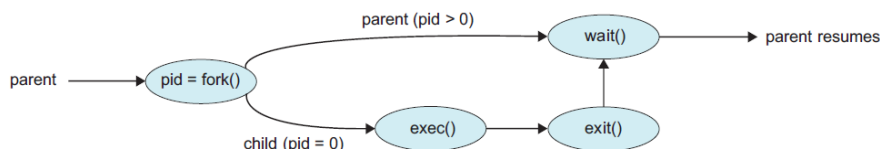


图 2-1 `exec_demo.c` 程序的执行过程

4、给出实验分析与结论

5、参考程序清单

清单 2-1 创建进程

```
#include <stdio.h>
```



```
#include <stdlib.h>
main()
{
    int x;
    srand((unsigned)time(NULL));
    while((x=fork())!=-1);
    if (x==0)
    {
        sleep(rand() % 2);
        printf("a\n");
    }
    else
    {
        sleep(rand() % 3);
        printf("b\n");
    }
    printf("c\n");
}
```

清单 2-2 子进程执行新任务

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid;
    /* fork a child process */
    pid = fork();
    if (pid < 0)
    { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0)
    { /* 子进程 */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* 父进程 */
        /* 父进程将一直等待，直到子进程运行完毕 */
        wait(NULL);
        printf("Child Complete\n");
    }
    return 0;
}
```

题目 A3: Linux 进程间通信

1、实验目的

Linux 系统的进程通信机构(IPC)允许在任意进程间大批量地交换数据,通过本实验,理解熟悉 Linux 支持的消息通信机制。

2、实验任务

消息的创建,发送和接收。

(1) 使用系统调用 `msgget()`, `msgsnd()`, `msgrcv()`及 `msgctl()`编制一长度为 1K 的消息的发送和接收程序。

(2) 观察参考程序,说明控制消息队列系统调用 `msgctl()`在此起什么作用?

3、背景知识

`fork` 系统调用的相关背景知识参见实验二。

UNIX/Linux 系统把信号量、消息队列和共享资源统称为进程间通信资源(IPC resource)。提供给用户的 IPC 资源是通过一组系统调用实现的。这组系统调用为用户态进程提供了以下三种服务:

- 用信号量对进程要访问的临界资源进行保护。
- 用消息队列在进程间以异步方式发送消息。
- 用一块预留出的内存区域供进程之间交换数据。

创建 IPC 资源的系统调用有:

- `semget()`—获得信号量的 IPC 标识符。
- `msgget()`—获得消息队列的 IPC 标识符。
- `shmget()`—获得共享内存的 IPC 标识符。

控制 IPC 资源的系统调用有:

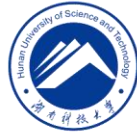
- `semctl()`—对信号量资源进行控制的函数。
- `msgctl()`—对消息队列进行控制的函数。
- `shmctl()`—对共享内存进行控制的函数。

上述函数为获得和设置资源的状态信息提供了一些命令。例如:

- `IPC_SET` 命令: 设置属主的用户标识符和组标识符。
- `IPC_STAT` 和 `IPC_INFO` 命令: 获得资源状态信息。
- `IPC_RMID` 命令: 释放这个资源。

操作 IPC 资源的系统调用有:

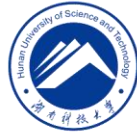
- `semop()`—获得或释放一个 IPC 信号量。 可以实现 P、V 操作
- `msgsnd()`—发送一个 IPC 消息。
- `msgrcv()`—接收一个 IPC 消息。



- `shmat()`—将一个 IPC 共享内存段添加到 进程的地址空间
- `shmdt()`——将 IPC 共享内存段从私有的地址空间剥离。

下面对部分系统调用说明如下：

- `msgget(key,flag)`
获得一个消息的描述符，该描述符指定一个消息队列以便用于其他系统调用。该函数使用头文件如下：
`#include <sys/types.h>`
`#include <sys/ipc.h>`
`#include <sys/msg.h>`
参数定义
`int msgget(key,flag)`
`key_t key;`
`int flag;`
语法格式：`msgqid=msgget(key,flag)`
其中：
`msgqid` 是该系统调用返回的描述符，失败则返回-1；
`flag` 本身由操作允许权和控制命令值相“或”得到。
如：`IPC_CREAT | 0400` 是否该队列应被创建；
`IPC_EXCL | 0400` 是否该队列的创建映象是互斥的；
- `msgsnd(id,msgp,size,flag)`
发送一消息。
该函数使用头文件如下：
`#include <sys/types.h>`
`#include <sys/ipc.h>`
`#include <sys/msg.h>`
参数定义：
`int msgsnd(id,msgp,size,flag)`
`int id,size,flag;`
`struct msgbuf *msgp;`
其中：`id` 是返回消息队列的描述符；`msgp` 是指向用户存储区的一个构造体指针，`size` 批示由 `msgp` 指向的数据结构中字符数组的长度，即消息的长度。这个数组的最大值由 `MSG_MAX` 系统可调用参数来确定。`flag` 规定当核心用尽内部缓冲空间时应执行的动作；若在标志范围 `flag` 中未设置 `IPC_NOWAIT` 位，则当该消息队列中的字节数超过一最大值时，或系统范围的消息数超过某一最大值时，调用 `msgsnd` 进程睡眠。若是设置 `IPC_NOWAIT`，则在此情况下，`msgsnd` 立即返回。
- `msgrcv(id,msgp,size,flag)`
接受一消息。
该函数调用使用头文件如下：
`#include <sys/types.h>`
`#include <sys/ipc.h>`
`#include <sys/msg.h>`
参数定义：
`int msgrcv(id,msgp,size,flag)`



```
int id,size,type,flag;
struct msgbuf *msgq;
struct msgbuf{ long mtype;char mtext[];};
语法格式:
```

```
count=msgrcv(id,msgp,size,type,flag)
```

其中: id 是消息描述符, msgp 是用来存放欲接收消息的拥护数据结构的地址; size 是 msgp 中数据数组的大小; type 是用户要读的消息类型:

type 为 0: 接收该项队列的第一个消息;

type 为正: 接收类型 type 的第一个消息;

type 为负: 接收小于或等于 type 绝对值的最低类型的第一个消息。

flag 规定倘若该队列无消息, 核心应当做什么事, 如果此时设置了 IPC_NOWAIT 标志, 则立即返回, 若在 flag 中设置了 MSG_NOERROR, 且所接收的消息大小大于 size, 核心截断所接收的消息。

count 是返回消息正文的字节数。

- msgctl(id,cmd,buf)

查询一个消息描述符的状态, 设置它的状态及删除一个消息描述符。

调用该函数使用头文件如下:

```
#include <sys/types.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/msg.h>
```

参数定义:

```
int msgctl(id,cmd,buf)
```

```
int id,cmd;
```

```
struct msgid_ds *buf;
```

其中: 函数调用成功时返回 0, 调用不成功时返回-1。

id 用来识别该消息的描述符; cmd 规定命令的类型。

IPC_STAT 将与 id 相关联的消息队列首标读入 buf。

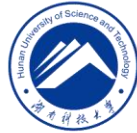
IPC_SET 为这个消息序列设置有效的用户和小组标识及操作允许权和字节的数量。

IPC_RMID 删除 id 的消息队列。

buf 是含有控制参数或查询结果的用户数据结构的地址。

附: msgid_ds 结构定义如下:

```
struct msgid_ds
{struct ipc_perm msg_perm;    /* 许可权结构 */
short pad1[7];               /* 由系统调用 */
ushort onsg_qnum;            /* 队列上消息数 */
ushort msg_qbytes;           /* 队列上最大字节数 */
ushort msg_lspid;            /* 最后发送消息的 PID */
ushort msg_lrpid;            /* 最后接收消息的 PID */
time_t msg_stime;            /* 最后发送消息的时间 */
time_t msg_rtime;            /* 最后接收消息的时间 */
time_t msg_ctime;            /* 最后更改时间 */
}
struct ipc_perm
```



```
{ushort uid;          /* 当前用户 id */
ushort gid;          /* 当前进程组 id */
ushort cuid;         /* 创建用户 id */
ushort cgid;         /* 创建进程组 id */
ushort mode;         /* 存取许可权 */
{short pad1;long pad2} /* 由系统调用 */
};
```

4、实验内容与步骤

消息的创建、发送和接收的程序设计

- (1) 为了便于操作和观察结果，用一个程序作为“引子”，先后 fork()两个子进程 SERVER 和 CLIENT，进行通信。
- (2) SERVER 端建立一个 key 为 75 的消息队列，等待其他进程发来的消息。当遇到类型为 1 的消息，则作为结束信号，取消该队列，并退出 SERVER。SERVER 每接收到一个消息后显示一句“(server) received”。
- (3) CLIENT 端使用 key 为 75 的消息队列，先后发送类型从 10 到 1 的消息，然后退出。最后的一个消息，即是 SERVER 端需要的结束信号。CLIENT 每发送一条消息后显示一句“(client) sent”。
- (4) 父进程在 SERVER 和 CLIENT 均退出后结束。

5、参考的程序清单

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/msg.h>
#include <sys/ipc.h>
#include <sys/wait.h>
#include <sys/ipc.h>
#include <stdlib.h>
#include <unistd.h>
#define MSGKEY 75
struct msgform
{
    long mtype;
    char mtext[300];
}msg;
int msgqid,i;
void CLIENT()
{
```



```
int i,result;
msgqid=msgget(MSGKEY,0777);
if(msgqid == -1)
{
    perror("client msgget");
    exit(1);
}
for (i=0;i<10;i++)
{

    printf("(client) sent \n");
    result = msgsnd(msgqid,&msg,300,0);
    if(result == -1)
    {
        perror("client msgsnd");
        exit(1);
    }
}
exit(0);
}
void SERVER()
{
    int i;
    int getsize = 0;

    msgqid=msgget(MSGKEY,0777);
    if(msgqid == -1)
    {
        perror("server msgget");
        exit(1);
    }
    for (i=0;i<10;i++)
    {
        getsize = msgrcv(msgqid,&msg,300,0,0);
        printf("(Server) recieved %d bytes\n",getsize);
    } ;
    msgctl(msgqid,IPC_RMID,0);
    exit(0);
}
void main()
{
    msgqid=msgget(MSGKEY,0777|IPC_CREAT);//777是权限控制bit, IPC_creat表示创建
    if(msgqid == -1)
```



```
{  
    perror("main msgget");  
}  
msg.mtype = 1;//必须是非负数，否则报错  
  
while((i=fork())!=-1);  
if(!i) SERVER();  
while((i=fork())!=-1);  
if(!i) CLIENT();  
wait(0);  
wait(0);  
}
```

6、实验结果与分析

(1)编译并运行程序，描述执行结果：“(client) sent”和“(server) received”各出现了几次，以什么顺序出现，解释出现这种结果的原因。

(2)请修改源代码，实现乒乓通信，即 client 发送第一条消息，server 接收到第一条消息后，client 再发送第二条消息，server 接收第二条消息，依此类推，表现为在端口中交替显示“(client) sent”和“(server) received”。

7、实验总结

题目 A4: Windows 线程的互斥与同步

1、实验目的

- (1) 回顾操作系统进程、线程的有关概念，加深对 Windows 线程的理解。
- (2) 了解互斥体对象，利用互斥与同步操作编写生产者-消费者问题的并发程序，加深对 P(即 semWait)、V(即 semSignal)原语以及利用 P、V 原语进行进程间同步与互斥操作的理解。

2、实验内容和步骤

生产者消费者问题

步骤 1: 创建一个“Win32 Consol Application”工程，然后拷贝清单 4-1 中的程序，编译成可执行文件。

步骤 2: 在“命令提示符”窗口运行步骤 1 中生成的可执行文件，列出运行结果。

步骤 3: 仔细阅读源程序，找出创建线程的 WINDOWS API 函数，回答下列问题：线程的第一个执行函数是什么（从哪里开始执行）？它位于创建线程的 API 函数的第几个参数中？

步骤 4: 修改清单 4-1 中的程序，调整生产者线程和消费者线程的个数，使得消费者数目大于生产者，看看结果有何不同。察看运行结果，从中你可以得出什么结论？

步骤 5: 修改清单 4-1 中的程序，按程序注释中的说明修改信号量 EmptySemaphore 的初始化方法，看看结果有何不同。

步骤 6: 根据步骤 4 的结果，并查看 <https://learn.microsoft.com/en-us/windows/win32/api/>，回答下列问题：

- 1) CreateMutex 中有几个参数，各代表什么含义。
- 2) CreateSemaphore 中有几个参数，各代表什么含义，信号量的初值在第几个参数中。
- 3) 程序中 P、V 原语所对应的实际 Windows API 函数是什么，写出这几条语句。
- 4) CreateMutex 能用 CreateSemaphore 替代吗？尝试修改程序 4-1，将信号量 Mutex 完全用 CreateSemaphore 及相关函数实现。写出要修改的语句。

3、 给出实验分析与结论

4、 参考程序清单

清单 4-1 生产者消费者问题

```
#include <windows.h>
#include <iostream>
```

```
const unsigned short SIZE_OF_BUFFER = 2; //缓冲区长度
unsigned short ProductID = 0; //产品号
unsigned short ConsumeID = 0; //将被消耗的产品号
unsigned short in = 0; //产品进缓冲区时的缓冲区下标
unsigned short out = 0; //产品出缓冲区时的缓冲区下标
```




```
int buffer[SIZE_OF_BUFFER];    //缓冲区是个循环队列
bool p_continue = true;        //控制程序结束
HANDLE Mutex;                  //用于线程间的互斥
HANDLE FullSemaphore;          //当缓冲区满时迫使生产者等待
HANDLE EmptySemaphore;         //当缓冲区空时迫使消费者等待

DWORD WINAPI Producer(LPVOID); //生产者线程
DWORD WINAPI Consumer(LPVOID); //消费者线程

int main()
{
    //创建各个互斥信号
    //注意, 互斥信号量和同步信号量的定义方法不同, 互斥信号量调用的是 CreateMutex 函数, 同步信号量调用的是 CreateSemaphore 函数, 函数的返回值都是句柄。
    Mutex = CreateMutex(NULL, FALSE, NULL);
    EmptySemaphore = CreateSemaphore(NULL, 0, SIZE_OF_BUFFER - 1, NULL);
    FullSemaphore = CreateSemaphore(NULL, 0, SIZE_OF_BUFFER, NULL);

    //将上句做如下修改, 看看结果会怎样
    //EmptySemaphore = CreateSemaphore(NULL, 0, SIZE_OF_BUFFER - 1, NULL);
    FullSemaphore = CreateSemaphore(NULL, 0, SIZE_OF_BUFFER, NULL);

    //调整下面的数值, 可以发现, 当生产者个数多于消费者个数时,
    //生产速度快, 生产者经常等待消费者; 反之, 消费者经常等待
    const unsigned short PRODUCERS_COUNT = 3; //生产者的个数
    const unsigned short CONSUMERS_COUNT = 1; //消费者的个数

    //总的线程数
    const unsigned short THREADS_COUNT = PRODUCERS_COUNT + CONSUMERS_COUNT;

    HANDLE hThreads[THREADS_COUNT]; //各线程的 handle
    DWORD producerID[PRODUCERS_COUNT]; //生产者线程的标识符
    DWORD consumerID[CONSUMERS_COUNT]; //消费者线程的标识符

    //创建生产者线程
    for (int i = 0; i < PRODUCERS_COUNT; ++i) {
        hThreads[i] = CreateThread(NULL, 0, Producer, NULL, 0, &producerID[i]);
        if (hThreads[i] == NULL) return -1;
    }
    //创建消费者线程
    for (i = 0; i < CONSUMERS_COUNT; ++i) {
```



```
hThreads[PRODUCERS_COUNT+i]=CreateThread(NULL,0,Consumer,NULL,0,&consumerID[i]);
```

```
    if (hThreads[i]==NULL) return -1;
}
```

```
while(p_ccontinue){
    if(getchar()){ //按回车后终止程序运行
        p_ccontinue = false;
    }
}
return 0;
}
```

//生产一个产品。简单模拟了一下，仅输出新产品的 ID 号

```
void Produce()
{
    std::cout << std::endl<< "Producing " << ++ProductID << " ... ";
    std::cout << "Succeed" << std::endl;
}
```

//把新生产的产品放入缓冲区

```
void Append()
{
    std::cerr << "Appending a product ... ";
    buffer[in] = ProductID;
    in = (in+1)%SIZE_OF_BUFFER;
    std::cerr << "Succeed" << std::endl;

    //输出缓冲区当前的状态
    for (int i=0;i<SIZE_OF_BUFFER;++i){
        std::cout << i <<": " << buffer[i];
        if (i==in) std::cout << " <-- 生产";
        if (i==out) std::cout << " <-- 消费";
        std::cout << std::endl;
    }
}
```

//从缓冲区中取出一个产品

```
void Take()
{
    std::cerr << "Taking a product ... ";
    ConsumeID = buffer[out];
    buffer[out] = 0;
```



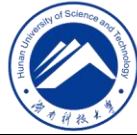
```
out = (out+1)%SIZE_OF_BUFFER;
std::cerr << "Succeed" << std::endl;

//输出缓冲区当前的状态
for (int i=0;i<SIZE_OF_BUFFER;++i){
    std::cout << i <<": " << buffer[i];
    if (i==in) std::cout << " <-- 生产";
    if (i==out) std::cout << " <-- 消费";
    std::cout << std::endl;
}
}

//消耗一个产品
void Consume()
{
    std::cout << "Consuming " << ConsumeID << " ... ";
    std::cout << "Succeed" << std::endl;
}

//生产者
DWORD WINAPI Producer(LPVOID lpPara)
{
    while(p_continue){
        WaitForSingleObject(EmptySemaphore,INFINITE); //p(empty);
        WaitForSingleObject(Mutex,INFINITE); //p(mutex);
        Produce();
        Append();
        Sleep(1500);
        ReleaseMutex(Mutex); //V(mutex);
        ReleaseSemaphore(FullSemaphore,1,NULL); //V(full);
    }
    return 0;
}

//消费者
DWORD WINAPI Consumer(LPVOID lpPara)
{
    while(p_continue){
        WaitForSingleObject(FullSemaphore,INFINITE);//P(full);
        WaitForSingleObject(Mutex,INFINITE); //P(mutex);
        Take();
        Consume();
        Sleep(1500);
    }
}
```



```
ReleaseMutex(Mutex);           //V(mutex);
ReleaseSemaphore(EmptySemaphore,1,NULL);    //V(empty);
}
return 0;
}
```

题目 A5: Windows 内存管理

1、实验目的

了解 Windows 的内存结构和虚拟内存的管理，理解进程的虚拟内存空间和物理内存的映射关系。加深对操作系统内存管理、虚拟存储管理等理论知识的理解。

2、背景知识

请参考教材《操作系统精髓与设计原理（中文第九版）》P240 8.5 Windows 内存管理。

Windows Xp 是 32 位的操作系统，它使计算机 CPU 可以用 32 位地址对 32 位内存块进行操作。内存中的每一个字节都可以用一个 32 位的指针来寻址。这样，最大的存储空间就是 2^{32} 字节或 4G 字节。这样，在 Windows 下运行的每一个应用程序最大可能占有 4GB 大小的空间。

然而，实际上每个进程一般不会占有 4GB 内存。Windows 在幕后将虚拟内存 (virtual memory, VM) 地址映射到了各进程的物理内存地址上。而所谓物理内存是指计算机的 RAM 和由 Windows 分配到用户驱动器根目录上的换页文件。

在 Windows 环境下，4GB 的虚拟地址空间被划分成两个部分：低端 2GB 提供给进程使用，高端 2GB 提供给系统使用。这意味着用户的应用程序代码，包括 DLL 以及进程使用的各种数据等，都装在用户进程地址空间内(低端 2GB)。

如何管理虚拟地址空间？Windows 内存管理器维持了一种数据结构，以记住哪些虚拟地址已被保留在进程的地址空间中，而哪些还没有。这些数据结构称为虚拟地址描述符 (virtual address descriptors, VAD)。对于每个进程，Windows 内存管理器维护了一组 VAD，以描述进程地址空间的状态。VAD 被构造成一棵自平衡二叉树(self-balancing binary tree)，以使查找高效。

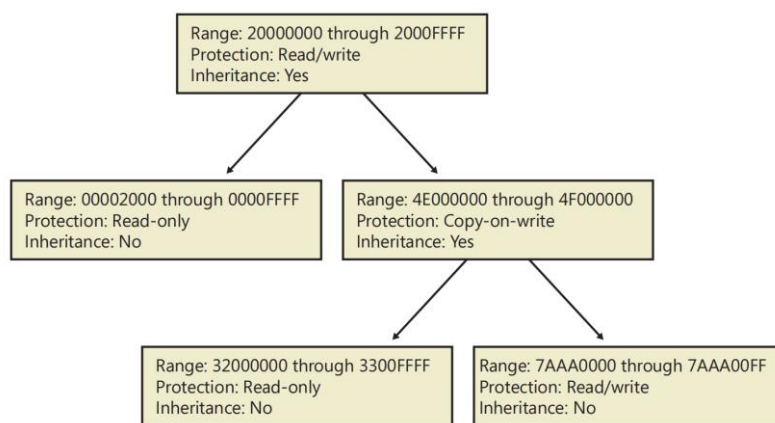


图 5-1 虚拟地址描述符

虚拟地址空间被分为很多 region，一个 region 是一个虚拟地址连续的区域，可以包含很多页，一个 region 由一个数据结构 VAD 进行描述和管理。VAD 是一个结构体，包含 region 的范围、大小、状态、权限等信息。图 5-1 摘自《Windows Internals (第 6 版)》，描绘了一棵由 committed 和 reserved 状态的 VAD 组成的二叉树，如果一个页的地址范围不在树中，则它的状态是 free。

状态方面，VAD 规定了一个 region 可以处于以下三种状态(state)之一：

●空置(free)：没有对应的 VAD。操作系统认为这段虚拟地址空间对于该进程而言，当前是**不可用的**，如果进程访问这个 region 中的地址，将引发一个异常。在未来，操作系统可以根据需要，为 free region 分配一个 VAD，将转换为 reserved 或 committed。

●提交(committed)：操作系统认为这段虚拟地址空间对于该进程而言，当前是**可用的**，如果进程访问这个 region 中的地址，操作系统要么从物理内存中找到进程需要的内容，要么通过交换从硬盘中找到进程需要的内容。

●保留(reserved)：堆和栈是动态增长的，而且要求虚拟地址连续。当栈比较小时，可以将其后续地址所在的 region 宣布为 reserved，从而为栈保留一定范围的虚拟地址以便将来变大后使用。如果进程访问 reserved region 中的地址，将引发一个异常。

Reserved 可以理解为从 free 到 committed 的中间阶段，通过将 free-committed 这一个步骤，拆分为 free-reserved 和 reserved-committed 两个步骤，将“提交”延迟到被进程需要的最后时刻，可以降低内存使用量，同时拥有虚拟地址连续的便利性。

权限方面，VAD 规定了进程可在内存中进行何种类型的操作。例如，进程不能在只有 PAGE_READONLY 权限的区域上进行写操作或执行程序；也不能在只有 PAGE_EXECUTE 权限的区域里进行读、写操作。而具有 PAGE_NOACCESS 权限的特殊区域，不允许进程进行任何操作。

在进程装入之前，整个虚拟内存的地址空间都被设置为只有 PAGE_NOACCESS 权限的 free 区域。当系统装入进程代码和数据后，才将内存地址的空间标记为已调配区或保留区，并将诸如 EXECUTE、READWRITE 和 READONLY 的权限与这些区域相关联。

Windows 提供了一整套 API 来访问 VAD 及相关管理结构，如表 5-1 所示。

表 5-1 虚拟内存的 API

API 名称	描述
VirtualQueryEx()	通过填充 MEMORY_BASIC_INFORMATION 结构检测进程内虚拟内存的区域
VirtualAlloc()	保留或调配进程的部分虚拟内存，设置分配和保护标志
VirtualFree()	释放或收回应用程序使用的部分虚拟地址
VirtualProtect()	改变虚拟内存区域保护规范
VirtualLock()	防止系统将虚拟内存区域通过系统交换到页面文件中
VirtualUnlock()	释放虚拟内存的锁定区域，必要时，允许系统将其交换到页面文件中

其中 Virtual QueryEX() API 填充的 MEMORY_BASIC_INFORMATION 结构，具体内容如表 5-2 所示，其大部分信息都搜集自 VAD。

表 5-2 MEMORY_BASIC_INFORMATION 结构的成员

成员名称	目的
PVOID BaseAddress	虚拟内存区域开始处的指针
PVOID AllocationBase	如果这个特定的区域为子分配区的话，则为虚拟内存外面区域的指针；否则此值与 BaseAddress 相同
DWORD AllocationProtect	虚拟内存最初分配区域的保护属性。其可能值包括： PAGE_NOACCESS, PAGE_READONLY, PAGE_READWRITE 和 PAGE_EXECUTE_READ
DWORD RegionSize	虚拟内存区域的字节数
DWORD State	区域的当前分配状态。其可能值为 MEM_COMMIT, MEM_FREE 和 MEM_RESERVE
DWORD Protect	虚拟内存当前区域的保护属性。可能值与 AllocationProtect 成员



	的相同
DWORD Type	虚拟内存区域中出现的页面类型。可能值为 MEM_IMAGE, MEM_MAPPED 和 MEM_PRIVATE

MEMORY_BASIC_INFORMATION 结构, State 项表明这些区域是否为 free 区、committed 或 reserved 区; Protect 项则说明了 Windows 系统为这些区域添加了何种访问保护; Type 项则表明这些区域是可执行镜像 image、内存映射文件 mapped 还是简单的私有内存 private。

提供虚拟内存分配功能的是 VirtualAlloc() API。该 API 支持用户向系统要求新的虚拟内存或改变已分配内存的当前状态。用户若想通过 VirtualAlloc() 函数使用虚拟内存, 可以采用两种方式通知系统:

- 1) 简单地将内存内容保存在地址空间内;
- 2) 请求系统返回带有物理存储区 (RAM 的空间或换页文件) 的部分地址空间。

用户可以用 flAllocationType 参数 (commit 和 reserve) 来定义这些方式, 用户可以通知 Windows 按只读、读写、不可读写、执行或特殊方式来处理新的虚拟内存。

与 VirtualAlloc() 函数对应的是 VirtualFree() 函数, 其作用是释放虚拟内存中的已调配页或保留页。用户可利用 dwFreeType 参数将已调配页修改成保留页属性。

VirtualProtect() 是 VirtualAlloc() 的一个辅助函数, 利用它可以改变虚拟内存区的保护规范。

3、 实验内容和步骤

了解和检测进程的虚拟内存空间。

步骤 1: 创建一个“Win32 Consol Application”工程, 拷贝清单 5-1 中的程序, 编译成可执行文件。

步骤 2: 在 VC 的工具栏单击“Execute Program”(执行程序) 按钮, 或者按 Ctrl + F5 键, 或者在“命令提示符”窗口运行步骤 1 中生成的可执行文件。

步骤 3: 根据运行结果, 回答下列问题。

为什么显示的地址都以 3 个 0 结尾: _____

为什么没有 00010000 之前的地址: _____

为什么没有 7fff0000 之后的地址: _____

虚拟内存每页大小为: _____

应用程序有权限读的最小地址: _____

应用程序有权限写的最小地址: _____

应用程序有权限读的最大地址: _____

应用程序有权限写的最大地址: _____

当前计算机的实际内存大小为: _____

通过网络搜索自主学习, 请自行组织语言, 指出 committed 区、reserved 区、free 区三者之间的区别: _____。

按 committed、reserved、free 等三种虚拟地址空间分别记录实验数据。其中“描述”是指对该组数据的简单描述, 例如, 对下列一组数据:

00010000 – 00012000 <8.00KB> Committed, READWRITE, Private

可描述为: 具有 READWRITE 权限的已提交私有内存区。

将系统当前的自由区 (free) 虚拟地址空间按表 5-3 格式记录。

表 5-3 实验记录

地址	大小	虚拟地址空间类型	访问权限	描述
----	----	----------	------	----

		free		
--	--	------	--	--

提示：详细记录实验数据在实验活动中是必要的，但想想是否可以简化记录的办法？

将系统当前的已调配区 (committed) 虚拟地址空间按表 5-4 格式记录。

表 5-4 实验记录

地址	大小	虚拟地址空间类型	访问权限	描述
		committed		

将系统当前的保留区 (reserved) 虚拟地址空间按表 5-5 格式记录。

表 5-5 实验记录

地址	大小	虚拟地址空间类型	访问权限	描述
		reserved		

4、 实验结论

网络搜索自主学习，简单描述 windows 进程的虚拟内存管理方案。

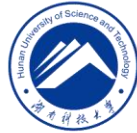
5、 程序清单

清单 5-1 了解和检测进程的虚拟内存空间

```
// 工程 vmwalker
#include <windows.h>
#include <iostream>
#include <shlwapi.h>
#include <iomanip>
#pragma comment(lib, "Shlwapi.lib")

// 以可读方式对用户显示保护的辅助方法。
// 保护标记表示允许应用程序对内存进行访问的类型
// 以及操作系统强制访问的类型
inline bool TestSet(DWORD dwTarget, DWORD dwMask)
{
    return ((dwTarget & dwMask) == dwMask);
}

# define SHOWMASK(dwTarget, type) \
if (TestSet(dwTarget, PAGE_ ## type) ) \
    {std :: cout << ", " << #type; }
```



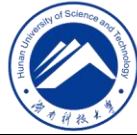
```
void ShowProtection(DWORD dwTarget)
{
    SHOWMASK(dwTarget, READONLY);
    SHOWMASK(dwTarget, GUARD);
    SHOWMASK(dwTarget, NOCACHE);
    SHOWMASK(dwTarget, READWRITE);
    SHOWMASK(dwTarget, WRITECOPY);
    SHOWMASK(dwTarget, EXECUTE);
    SHOWMASK(dwTarget, EXECUTE_READ);
    SHOWMASK(dwTarget, EXECUTE_READWRITE);
    SHOWMASK(dwTarget, EXECUTE_WRITECOPY);
    SHOWMASK(dwTarget, NOACCESS);
}

// 遍历整个虚拟内存并对用户显示其属性的工作程序的方法
void WalkVM(HANDLE hProcess)
{
    // 首先，获得系统信息
    SYSTEM_INFO si;
    ::ZeroMemory(&si, sizeof(si));
    ::GetSystemInfo(&si);

    // 分配要存放信息的缓冲区
    MEMORY_BASIC_INFORMATION mbi;
    ::ZeroMemory(&mbi, sizeof(mbi));

    // 循环整个应用程序地址空间
    LPCVOID pBlock = (LPVOID) si.lpMinimumApplicationAddress;
    while (pBlock < si.lpMaximumApplicationAddress)
    {
        // 获得下一个虚拟内存块的信息
        if (::VirtualQueryEx(
            hProcess,           // 相关的进程
            pBlock,             // 开始位置
            &mbi,               // 缓冲区
            sizeof(mbi) == sizeof(mbi)) // 大小的确认
        {
            // 计算块的结尾及其大小
            LPCVOID pEnd = (PBYTE) pBlock + mbi.RegionSize;
            TCHAR szSize[MAX_PATH];
            ::StrFormatByteSize(mbi.RegionSize, szSize, MAX_PATH);

            // 显示块地址和大小
        }
    }
}
```



```
std :: cout.fill ('0') ;
std :: cout
    << std :: hex << std :: setw(8) << (DWORD) pBlock
    << "_"
    << std :: hex << std :: setw(8) << (DWORD) pEnd
    << (:: strlen(szSize)==7? " (" : " (") << szSize
    << ")" " ;

// 显示块的状态
switch(mbi.State)
{
    case MEM_COMMIT :
        std :: cout << "Committed" ;
        break;
    case MEM_FREE :
        std :: cout << "Free" ;
        break;
    case MEM_RESERVE :
        std :: cout << "Reserved" ;
        break;
}
// 显示保护
if(mbi.Protect==0 && mbi.State!=MEM_FREE)
{    mbi.Protect=PAGE_READONLY;    }
ShowProtection(mbi.Protect);
// 显示类型
switch(mbi.Type){
    case MEM_IMAGE :
        std :: cout << ", Image" ;
        break;
    case MEM_MAPPED:
        std :: cout << ", Mapped";
        break;
    case MEM_PRIVATE :
        std :: cout << ", Private" ;
        break;
}
// 检验可执行的影像
TCHAR szFilename [MAX_PATH] ;
if (:: GetModuleFileName (
    (HMODULE) pBlock,        // 实际虚拟内存的模块句柄
    szFilename,              // 完全指定的文件名称
    MAX_PATH)>0)              // 实际使用的缓冲区大小
```



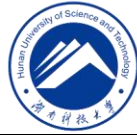
```
{
    // 除去路径并显示
    :: PathStripPath(szFilename) ;
    std :: cout << " , Module: " << szFilename;
}
std :: cout << std :: endl;
// 移动块指针以获得下一个块
pBlock = pEnd;
}
}
}

void ShowVirtualMemory()
{
    // 首先，让我们获得系统信息
    SYSTEM_INFO si;
    :: ZeroMemory(&si, sizeof(si) ) ;
    :: GetSystemInfo(&si) ;
    // 使用外壳辅助程序对一些尺寸进行格式化
    TCHAR szPageSize[MAX_PATH];
    :: StrFormatByteSize(si.dwPageSize, szPageSize, MAX_PATH) ;
    DWORD dwMemSize = (DWORD)si.lpMaximumApplicationAddress -
        (DWORD) si.lpMinimumApplicationAddress;
    TCHAR szMemSize [MAX_PATH] ;
    :: StrFormatByteSize(dwMemSize, szMemSize, MAX_PATH) ;
    // 将内存信息显示出来
    std :: cout << "Virtual memory page size: " << szPageSize << std :: endl;

    std :: cout.fill ('0') ;
    std :: cout << "Minimum application address: 0x"
        << std :: hex << std :: setw(8)
        << (DWORD) si.lpMinimumApplicationAddress
        << std :: endl;
    std :: cout << "Maximum application address: 0x"
        << std :: hex << std :: setw(8)
        << (DWORD) si.lpMaximumApplicationAddress
        << std :: endl;

    std :: cout << "Total available virtual memory: "
        << szMemSize << std :: endl ;
}

void main()
{
```



```
//显示虚拟内存的基本信息
ShowVirtualMemory();
// 遍历当前进程的虚拟内存
::WalkVM(::GetCurrentProcess());
}
```

B 组（综合题）

题目 B1：模拟实现银行家算法

1、实验目的

- (1) 进一步了解进程的并发执行。
- (2) 加强对进程死锁的理解，理解安全状态与不安全状态的概念。
- (3) 掌握使用银行家算法避免死锁问题。

2、实验基本知识及原理

（1）基本概念

死锁：多个进程在执行过程中，因为竞争资源会造成相互等待的局面。如果没有外力作用，这些进程将永远无法向前推进。此时称系统处于死锁状态或者系统产生了死锁。

安全序列：系统按某种顺序并发进程，并使它们都能达到获得最大资源而顺序完成的序列为安全序列。

安全状态：能找到安全序列的状态称为安全状态，安全状态不会导致死锁。

不安全状态：在当前状态下不存在安全序列，则系统处于不安全状态。

（2）银行家算法

银行家算法顾名思义是来源于银行的借贷业务，一定数量的本金要满足多个客户的借贷周转，为了防止银行家资金无法周转而倒闭，对每一笔贷款，必须考察其是否能限期归还。

在操作系统中研究资源分配策略时也有类似问题，系统中有限的资源要供多个进程使用，必须保证得到的资源的进程能在有限的时间内归还资源，以供其它进程使用资源。如果资源分配不当，就会发生进程循环等待资源，则进程都无法继续执行下去的死锁现象。

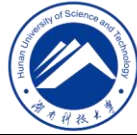
当一个进程提出资源申请时，银行家算法执行下列步骤以决定是否向其分配资源：

- 1) 检查该进程所需要的资源是否已超过它所宣布的最大值。
- 2) 检查系统当前是否有足够资源满足该进程的请求。
- 3) 系统试探着将资源分配给该进程，得到一个新状态。
- 4) 执行安全性算法，若该新状态是安全的，则分配完成；若新状态是不安全的，则恢复原状态，阻塞该进程。

3、实验内容

本实验的内容是要通过编写和调试一个模拟系统动态分配资源的银行家算法程序，有效地避免死锁发生。具体要求如下：

- (1) 初始化时让系统拥有一定的资源；
- (2) 用键盘输入的方式允许进程动态申请资源；
- (3) 如果试探分配后系统处于安全状态，则修改系统的资源分配情况，正式分配资源；
- (4) 如果试探分配后系统处于不安全状态，则提示不能满足请求，恢复原状态并阻塞该进程。



4、 银行家算法中的数据结构与流程图

(1) 数据结构 (详见教材 P177):

资源总量向量 $Resource$, m 维, 表示 m 种资源的总量。

可用资源向量 $Available$, m 维, 表示未分配的各种可用资源数量。

需求矩阵 $Claim$, $n*m$ 矩阵, 表示 n 个进程对 m 类资源的最大需求。

分配矩阵 $Allocation$, $n*m$ 矩阵, 表示 n 个进程已分配的各种资源数。

(2) 算法的程序流程图: 请补充画出算法程序流程图。

5、 银行家算法编程实现

根据银行家算法及其数据结构, 利用 C/C++ 语言进行编程实现。

题目 B2：模拟实现页面置换算法

1、实验目的

通过对页面、页表、地址转换和页面置换过程的模拟，加深对虚拟页式内存管理系统的页面置换原理和实现过程的理解。

2、实验基本知识及原理

需要调入新页面时，选择内存中哪个物理页面被置换，称为置换策略。页面置换算法的目标：把未来不再使用的或短期内较少使用的页面调出，通常应在局部性原理指导下依据过去的统计数据预测，减少缺页次数。

教材给出的常用的页面置换算法包括：

- 1) 最佳置换算法(OPT)：置换时淘汰“未来不再使用的”或“在离当前最远位置上出现的”页面。
- 2) 先进先出置换算法(FIFO)：置换时淘汰最先进入内存的页面，即选择驻留在内存时间最长的页面被置换。
- 3) 最近最久未用置换算法(LRU)：置换时淘汰最近一段时间最久没有使用的页面，即选择上次使用距当前最远的页面淘汰
- 4) 时钟算法(Clock)：也称最近未使用算法(NRU, Not Recently Used)，它是 LRU 和 FIFO 的折衷。

3、实验内容

教材《操作系统精髓与设计原理（中文第九版）》P227 图 8.14 给出了一个页框数=3，页数=5，页地址顺序（又称页地址流）={ 2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2}的例子，来说明以上 4 种页面置换算法。

请使用 C 语言编程实现 OPT、FIFO、LRU、Clock 算法，得到和教材图 8.14 相同的结果。

思考：

- 如果增加页框数，会对缺页率产生什么影响？
- 为什么一般情况下，LRU 具有比 FIFO 更好的性能？

题目 B3：模拟实现磁盘调度算法

1、实验目的

- (1) 了解磁盘结构以及磁盘上数据的组织方式。
- (2) 掌握磁盘访问时间的计算方式。
- (3) 掌握常用磁盘调度算法及其相关特性。

2、实验基本知识及原理

(1) 磁盘数据的组织

磁盘上每一条物理记录都有唯一的地址，该地址包括三个部分：磁头号（盘面号）、柱面号（磁道号）和扇区号。给定这三个量就可以唯一地确定一个地址。

(2) 磁盘访问时间的计算方式

磁盘在工作时以恒定的速率旋转。为保证读或写，磁头必须移动到所要求的磁道上，当所要求的扇区的开始位置旋转至磁头下时，开始读或写数据。对磁盘的访问时间包括：寻道时间、旋转延迟时间和传输时间。

(3) 磁盘调度算法

磁盘调度的目的是要尽可能降低磁盘的寻道时间，以提高磁盘 I/O 系统的性能。

先进先出算法：按访问请求到达的先后次序进行调度。

最短服务时间优先算法：优先选择使磁头臂从当前位置开始移动最少的磁盘 I/O 请求进行调度。

SCAN（电梯算法）：要求磁头臂先沿一个方向移动，并在途中满足所有未完成的请求，直到它到达这个方向上的最后一个磁道，或者在这个方向上没有别的请求为止，后一种改进有时候称作 LOOK 策略。然后倒转服务方向，沿相反方向扫描，同样按顺序完成所有请求。

C-SCAN（循环扫描）算法：在磁盘调度时，把扫描限定在一个方向，当沿某个方向访问到最后一个磁道时，磁头臂返回到磁盘的另一端，并再次开始扫描。

3、实验内容

本实验通过编程模拟实现几种常见的磁盘调度算法。

- (1) 测试数据：参见教材 P319-320，测试结果参见教材表 11.2。
- (2) 使用 C 语言编程实现 FIFO、SSTF、SCAN、C-SCAN 算法。

题目 B4：使用信号量解决读者写者问题

1、实验目的

(1) 回顾操作系统进程、线程的有关概念，针对经典的同步、互斥、死锁与饥饿问题进行并发程序设计。

(2) 了解互斥体对象，利用互斥与同步操作编写读者写者问题的并发程序，加深对 P(即 semWait)、V(即 semSignal)原语以及利用 P、V 原语进行进程间同步与互斥操作的理解。

2、实验基本知识及原理

(1) 掌握与并发控制相关的 Windows API 函数。

(2) UNIX/Linux 系统把信号量、消息队列和共享资源统称为进程间通信资源(IPC resource)。提供给用户的 IPC 资源是通过一组系统调用实现的，常见的有 semget(), semop(), 及 semctl()等信号量系统调用，具体如何使用请自行查询有关资料。

3、实验内容和要求

熟悉 P、V 原语对应的实际 API 函数，并参考教材中读者写者问题的算法原理，尝试利用 Windows API 函数或 Linux API 实现第一类读者写者问题（读者优先）。

题目 B5：使用信号量解决哲学家进餐问题

1、实验目的

(1) 回顾操作系统进程、线程的有关概念，针对经典的同步、互斥、死锁与饥饿问题进行并发程序设计。

(2) 了解互斥体对象，加深对 P(即 `semWait`)、V(即 `semSignal`)原语以及利用 P、V 原语进行进程间同步与互斥操作的理解。

(3) 理解 Linux 支持的信息量机制，利用 IPC 的信号量系统调用编程实现哲学家进餐问题。

2、实验基本知识及原理

(1) 了解与并发控制相关的 Windows API。

(2) UNIX/Linux 系统把信号量、消息队列和共享资源统称为进程间通信资源(IPC resource)。提供给用户的 IPC 资源是通过一组系统调用实现的，常见的有 `semget()`, `semop()`, 及 `semctl()`等信号量系统调用，具体如何使用请自行查询有关资料。

3、实验内容和要求

使用系统调用 `semget()`, `semop()`, 及 `semctl()`编制一个哲学家进餐问题(参考教材 P183-184)的程序。思路：为了便于操作和观察结果，用主程序创建五个“叉子信号量”，然后先后 `fork()`五个哲学家子进程，使它们通过对信号量进行 P、V 操作并发地进行 thinking 与 eating。

题目 B6：模拟实现命令行解释器

1、实验目的

本实验主要目的在于进一步学会如何在 Linux 系统下使用进程相关的系统调用，了解 shell 工作的基本原理，自己动手为 Linux 操作系统设计一个命令接口。

2、实验基本知识及原理

本实验要使用创建子进程的 `fork()` 函数，执行新的命令的 `exec()` 系列函数，通常 shell 是等待子进程结束后再接受用户新的输入，这可以使用 `waitpid()` 函数。

3、设计内容及要求

要设计的 shell 类似于 `sh`、`bash`、`csh` 等，必须支持以下内部命令：

cd <目录> 更改当前的工作目录到另一个<目录>。如果<目录>未指定，输出当前工作目录。如果<目录>不存在，应当有适当的错误信息提示。这个命令应该也能改变 **PWD** 的环境变量。

environ 列出所有环境变量字符串的设置（类似于 Linux 系统下的 **env** 命令）。

echo <内容> 显示 echo 后的内容且换行

help 简短概要的输出你的 shell 的使用方法和基本功能。

jobs 输出 shell 当前的一系列子进程，必须提供子进程的命名和 PID 号。

quit,exit,bye 退出 shell。

提示：shell 的主体就是反复下面的循环过程

```
while(1){
    接收用户输入的命令行；
    解析命令行；
    if(用户命令为内部命令)
        直接处理；
    else if(用户命令为外部命令)
        创建子进程执行命令；
    else
        提示错误的命令；
}
```

题目 B7：模拟实现进程调度

1、实验目的

本实验主要目的在于加深对进程的概念、属性及进程调度过程/算法的理解。

2、实验基本知识及原理

调度程序需在每次调度前，根据一定策略（如按优先级排序）更新和选择就绪队列中的进程。教材上给出了 6 种不同的调度策略：

先来先服务(FCFS): 每个进程就绪后，会加入就绪队列，当前正运行的进程停止执行时，选择就绪队列中存在时间最长的进程运行。

时间片轮转调度算法：所有就绪进程按先来先服务原则排成一个队列。每次调度时，将 CPU 分配给队首进程，并让其运行一个固定长度的时间片。若时间片用完进程仍未完成，则将其放回就绪队列末尾，CPU 重新分配给新的队首进程。此过程循环直至所有进程完成。

最短进程优先(SPN): 这是一个非抢占策略，其原则是下次选择预计处理时间最短的进程，因此，短进程将会越过长作业，跳到队列头。调度只发生在进程结束之时。

最短剩余时间(SRT): 在 SPN 中增加了抢占机制，调度程序总是选择预期剩余时间最短的进程，一个新进程加入就绪队列时，与当前正运行的进程相比，它可能具有更短的剩余时间。因此，只要新进程就绪，调度程序就可抢占当前正在运行的进程。

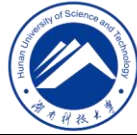
最高响应比优先(HRRN): 考虑下面的比值： $R = \frac{W+S}{S}$ ，式中， R 为响应比， W 为等待处理器的时间， S 为预计的服务时间，据此，调度规则如下：选择 R 值最大的就绪进程。

反馈法(FB): 如果无法预计服务时间，就不能使用 SPN、SRT 和 HRRN 方法，此时可以考虑另一种优先短作业的方法，即处罚运行时间较长的作业。换句话说，若不能获得剩余的执行时间，则关注已执行的时间。在不知道进程 A 长短的情况下，可以先让它运行 1 小时，如果它是很短的进程，应该已执行完了，否则，给它 2 小时，如果它是较短的进程，也应该执行完了，否则给 4 小时，如果还没执行完，A 就不是一个短进程，是一个长进程。通过不断的“反馈”，OS 可以逐步尝试出进程 A 的“长短”，从而可以对所有进程分类，分类以后，就可以执行“短进程优先”。

3、设计要求

(1)给出进程调度的算法描述。

(2)用 C 或 C++语言设计一个对 n 个并发进程进行调度的程序，每个进程由一个进程控制块(PCB)结构表示，该进程控制块应包括下述信息:进程标识 ID、进程需要的服务时间、进程已运行的时间、进程还需要运行的时间(初始值为进程服务时间，当进程运行结束时，为 0)、进程的状态 STATE(设每个进程处于运行 E(executing)、就绪 R(ready)和完成 F(finish)三



种状态之一，并假设起始状态都是就绪状态 R)。为简化起见，所有的时间均以时间片为单位。

(3)模拟调度程序应实现先来先服务 FCFS、时间片轮转 RR、最短进程优先 SPN、最短剩余时间优先 SRT、高响应比 HRRN、多级反馈队列 MLFQ 这 6 种算法，运行时选择一种，以利于各种方法的分析与比较。

(4)程序应能显示或打印各种进程状态和参数变化情况，便于观察分析。即要在每个时间片开始时，显示或打印每个进程的状态、优先级、还需要的服务时间、已运行时间等信息，并统计完成进程的周转时间及加权周转时间。

题目 B8: 模拟实现动态分区

1、实验目的

- (1) 了解操作系统的动态分区存储管理过程和方法。
- (2) 掌握动态分区存储管理的主要数据结构—空闲区表。
- (3) 加深理解动态分区存储管理中内存的分配和回收。
- (4) 掌握空闲区表中空闲区 3 种不同的放置策略的基本思想和实现过程。
- (5) 通过模拟程序实现动态分区存储管理。
- (6) 加深理解不同放置策略导致不同的内存分配效果。

2、实验基本知识及原理

1. 分区存储管理。

分区存储管理的基本原则是操作系统为每个作业分配一块满足大小要求的连续存储区,以连续存放作业的程序和数据。按照分区时机可以分为固定分区和动态分区两种方法。固定分区法是将内存划分为若干大小不等的区域,区域大小的划分由系统操作员或操作系统决定,分区一旦划分,则整个系统运行期间各个分区的大小和系统中总的分区数目就不再改变。

动态分区法中操作系统在作业执行前并不建立分区,而是在处理作业过程中按作业需要动态创建分区。

2. 动态分区中管理空闲区的数据结构

(1) 空闲区表

操作系统将所有的内存空闲区集中放到空闲区表中,当有作业请求空闲区时,操作系统在表中依次查找每一个空闲区,直到找到第一个满足作业大小要求的空闲区实施分配。表中保留分配后剩余的空闲区的大小,如果大小相等没有剩余空闲区,则将该表目从空闲区表中删除。

(2) 空闲区链

使用指针将所有的空闲区链接起来形成链表,称为空闲区链。分配时在链表中依次查找实验 13 动态分区管理算法空间满足大小要求的空闲区实施分配,如果有剩余空闲区则修改空闲区的大小,如果大小相等没有剩余空闲区则将该节点从链表中摘除并回收。

3. 空闲区的切割方式

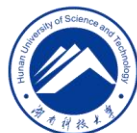
低地址切割: 剩余空闲区的首地址和大小均改变。

高地址切割: 剩余空闲区的首地址不变, 大小改变

4. 回收分区时空闲区邻接情况操作系统在回收内存时, 为了避免碎片, 往往采取相邻空闲区合并的算法, 以保证相邻的空闲区是一个整体。回收内存可能出现的邻接情况有 4 种(见下图):

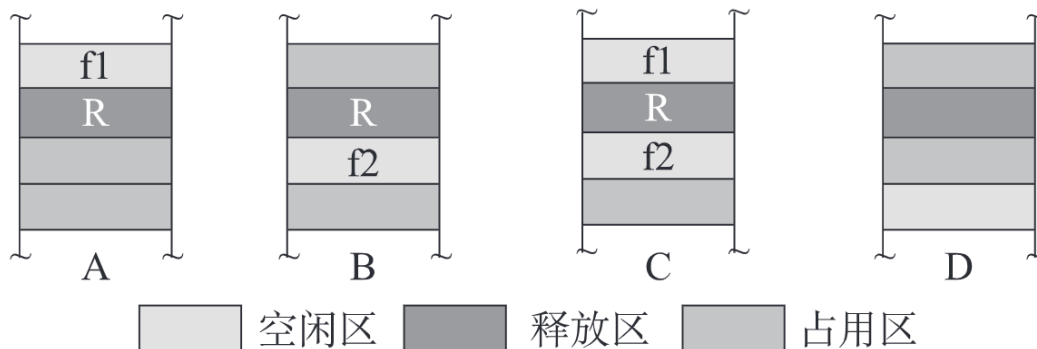
(1) 有上邻无下邻。回收分区与上邻合并, 上邻空闲区首地址不变, 上邻空闲区大小改为 $f1+f$ 。

(2) 无上邻有下邻。回收分区与下邻合并, 下邻空闲区首地址改为 f , 下邻空闲区大小改为 $f+f2$ 。



(3) 有上、下邻。回收分区与上、下邻合并，上邻空闲区首地址不变，上邻空闲区大小改为 $f1+f2$ ，在空闲区表中删除下邻空闲区表目。

(4) 无上、下邻。在空闲区表中增加一个新的空闲区表目。



5. 空闲区表的放置策略

- (1) 首次适应算法：空闲区按照物理地址由低到高的顺序排列。
- (2) 最佳适应算法：空闲区按照空间大小由小到大的顺序排列。
- (3) 最坏适应算法：空闲区按照空间大小由大到小的顺序排列。

6. 本次实验中使用的空闲区结构的组成
本次实验中使用的空闲区结构由空闲区首地址、空闲区大小、空闲区状态组成。

3、设计要求

(1) 假设内存空间大小为 1024KB，根据程序给出的初始空闲区表信息，画出内存的初始状态示意图。

(2) 在此基础上，系统依次执行了以下操作，画出内存空间的变化情况。

- ①作业 1 向系统申请 50KB。
- ②系统释放作业 2，首地址 250KB，大小 200KB。
- ③系统释放作业 3，首地址 450KB，大小 150KB。
- ④作业 4 向系统申请 200KB。
- ⑤系统释放作业 1（首地址 80KB，大小 50KB）。
- ⑥系统释放作业 4（首地址 150KB，大小 200KB）。
- ⑦作业 5 向系统申请 600KB。

(3) 本题中共执行了 3 次申请内存操作，每次操作申请的内存大小与空闲区的大小情况如何？

(4) 本题中共执行了 4 次释放内存操作，每次操作的释放区与空闲区的邻接情况如何？

(5) 对分析结果与程序的运行结果进行验证，看看是否符合程序的运行结果。

题目 B9：模拟实现简单文件系统

1、实验目的

让学生自己动手设计一个简单的文件系统，进一步巩固操作系统的文件系统的理论知识。

2、实验内容

为 Linux/Unix 设计一个简单的二级文件系统，要求做到以下几点：

(1) 可以实现下列几条命令（至少 4 条）

Login	用户登录
Dir	列文件目录
Create	创建文件
Delete	删除文件
Open	打开文件
Close	关闭文件
Read	读文件
Write	写文件

(2) 列目录时要列出文件名、物理地址、保护码和文件长度

(3) 源文件可以进行读写保护

3、实验准备

(1) 首先应确定文件系统的数据结构：主目录、子目录及活动文件等。主目录和子目录都以文件的形式存放于磁盘，这样便于查找和修改；

(2) 用户创建的文件，可以编号存储于磁盘上。如 file0, file1, file2.....并以编号作为物理地址，在目录中进行登记。

4、实验指导

一、文件管理

要将文件存储在磁盘（带）上，必须为之分配相应的存储空间，这就涉及到对文件存储空间的管理；采取何种方式存储，又涉及到文件的物理结构；为了简化对文件的访问和共享，还应设置相应的用户文件描述表及文件表。

1、文件存储空间的管理

(1) 文件卷的组织

UNIX 中，把每个磁盘（带）看作是一个文件卷，每个文件卷上可存放一个具有独立目录结构的文件系统。一个文件卷包含许多物理块，并按块号排列如下图：

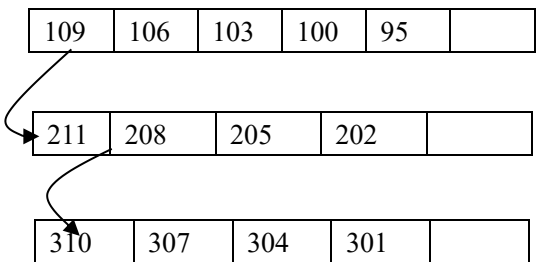
0#	1#	2#	3#	K#	K+1#	N#
----	----	----	----	---------	------	---------

其中,0#块用于系统引导或空闲,1#为超级块(superblock),存放文件卷的资源管理信息,如整个文件卷的盘块数、磁盘索引结点的盘块数、空闲盘块号栈及指针等。2#~K#存放磁盘索引结点。每个索引结点 64B,第 K+1#~N#存放文件数据。

(2) 空闲盘块的组织

UNIX 采用成组链接法组织空闲盘块。它将若干个空闲盘块划归一个组,将每组中所有盘块号存放在其前一组的第一个空闲盘块中,而第一组中所有空闲盘块号放入超级块的空闲盘块号栈中。

例： 超级块表



(3) 空闲盘块的分配与回收

内核要从文件系统中分配一盘块时,先检查超级块空闲盘块号栈是否已上锁。是则调用 sleep 睡眠,否则将超级块中空闲盘块栈栈顶盘块号分配出去。

回收时,若空闲盘块号栈未满,直接将回收盘块编号记入空闲盘块号栈中。若回收时栈已满,须先将栈中的所有空闲盘块号复制到新回收的盘块中,再将新回收盘块的编号作为新栈的栈底块号进栈。

2、文件的物理结构

UNIX 未采用传统的三种文件结构形式,而是将文件所占用盘块的盘块号,直接或间接地存放在该文件索引结点的地址项中。查找文件时,只需找到该文件的索引结点,便可直接或间接的寻址方式获得指定文件的盘块。

过程 bmap 可将逻辑文件的字节偏移量转换为文件的物理块号。先将字节偏移量转换为文件逻辑块号及块内偏移量,再把逻辑块号转换为文件的物理块号。

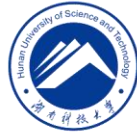
3、用户文件描述符表和文件表的管理

每个进程的 U 区中设置一张用户文件描述符表。只在首次打开文件时才需给出路径名。内核在该进程的用户文件描述符表中,分配一空项,取其在该表中的位移量作为文件描述符 fd(file descriptor)返还给用户。当用户再次访问该文件时,只需提供 fd,系统根据 fd 便可找到相应文件的内存索引结点。fd 表项的分配由 ufalloc 完成。

为了方便用户对文件进行读/写及共享,系统中设置了一张文件表。每个用户在打开文件时,都要在文件表中获得一表项,其中包含下述内容:

- f.flag: 文件标志,指示该文件打开是为了读或写;
- f.inode: 指向打开文件的内存索引结点指针;
- f.offset: 文件读写指针偏移值;
- f.count: 文件引用计数。

二、目录管理



UNIX 中，为了加速对文件目录的查找，将文件名和文件说明分开，由文件说明形成一个称为索引结点的数据结构，而相应的文件目录项则只由文件符号名和指向索引结点的指针构成。

对目录的管理应包括的功能有：

1、对索引结点的管理

每个文件都有一唯一的磁盘索引结点(`di_node`)。文件被打开后，还有一个内存索引结点(`i_node`)。创建一新文件时，就为之建立一个磁盘索引结点，以将文件的有关信息记入其中，并将用户提供的文件名和磁盘索引结点号一并组成一个新目录项，记入其父目录文件中。文件被撤消时，系统要回收该文件的磁盘索引结点，从其父目录中删除该目录项。随着文件的打开与关闭，系统还要为之分配和回收内存索引结点。

(1) 磁盘索引结点

磁盘索引结点中，包含有关文件的下述一系列信息：

文件模式 `di_mode`。可以是正规文件、目录文件、字符特别文件、块特别文件和管道文件等几种；

文件所有者用户标识符 `di_uid`。指拥有该文件的用户标识符；

同组用户标识符 `di_gid`。与拥有该文件的用户在同一小组的用户标识符；

文件长度 `di_size`。以字节计数的文件大小；

文件的联接计数 `di_nlink`。表明在本文件系统中所有指向该文件的文件名计数；

文件的物理地址 `di_addr`。`di_addr` 地址项共有 13 项，即 `di_addr(0)` 到 `di_addr(12)`，每个地址项占 3 字节；

文件的访问时间 `di_atime`。指文件最近被进程访问的时间；

文件的修改时间 `di_mtime`。指文件和索引结点最近被进程修改的时间；

文件的建立时间 `di_citime`。

(2) 内存索引结点

文件被打开后，系统为它在内存索引结点表区中建一内存索引结点，以方便用户和系统对文件的访问。其中，一部分信息是直接从磁盘索引结点拷贝过来的，如 `i_mode`、`i_uid`、`i_gid`、`i_size`、`i_addr`、`i_nlink` 等，并又增加了如下各信息：

索引结点编号 `i_number`。作为内存索引结点的标识符；

状态 `i_flag`。指示内存索引结点是否已上锁、是否有进程等待此 `i` 结点解锁、`i` 结点是否被修改、是否有最近被访问等标志；

引用计数 `i_count`。记录当前有几个进程正在访问此 `i` 结点，每当有进程访问此 `i` 结点时，对 `i_count+1`，退出-1；

设备号 `i_dev`。文件所属文 S 件系统的逻辑设备号；

前向指针 `i_forw`。Hash 队列的前向指针；

后向指针 `i_back`。Hash 队列的后向指针；

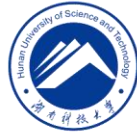
(3) 磁盘索引结点的分配与回收

分配过程 `ialloc`：当内核创建一新文件时，要为之分配一空闲磁盘 `i` 结点。如分配成功，便再分配一内存 `i` 结点。其过程如下：

检查超级块上锁否。由于超级块是临界资源，诸进程必须互斥地访问它，故在进入 `ialloc` 后，要先检查它是否已上锁，若是则睡眠等待；

检查 `i` 结点栈空否。若 `i` 结点栈中已无空闲结点编号，则应从盘中再调入一批 `i` 结点号进栈。若盘中已无空闲 `i` 结点，则出错处理，返回；

从空闲 `i` 结点编号栈中分配一 `i` 结点，并对它初始化、填写有关文件的属性；



分配内存 *i* 结点;

将磁盘 *i* 结点总数-1, 置超级块修改标志, 返回。

回收过程 *ifree*:

当删除文件时, 应回收其所占用的盘块及相应的磁盘 *i* 结点。具体有:

检查超级块上锁否。若是, 直接返回, 即不把本次回收的 *i* 结点号记入空闲 *i* 结点编号栈中;

检查 *i* 结点编号栈满否。若已满, 无法再装入新回收的 *i* 结点号, 立即返回, 若未滿, 便将回收的 *i* 结点编号进栈, 并使当前空闲结点数+1;

置超级块修改标志, 返回。

(4) 内存索引结点的分配与回收

分配过程 *iget*:

虽然 *iget* 用在打开文件时为之分配 *i* 结点, 但由于允许文件被共享, 因此, 如果一文件已被其他用户打开并有了内存 *i* 结点, 则此时只需将 *i* 结点中的引用计数+1。如果文件尚未被任何用户(进程)打开, 则由 *iget* 过程为该文件分配一内存 *i* 结点, 并调用 *bread* 过程将其磁盘 *i* 结点的内容拷贝到内存 *i* 结点中并进行初始化。

回收过程 *iput*:

进程要关闭某文件时, 须调用 *iput* 过程, 先对该文件内存 *i* 结点中的引用计数-1。若结果为 0, 便回收该内存 *i* 结点, 再对该文件的磁盘 *i* 结点中的连接计数减 1, 若其结果也为 0, 便删除此文件, 并回收分配给该文件的盘块和磁盘 *i* 结点。

2、构造目录 *make_node*

文件系统的一个基本功能是实现按名存取, 它通过文件目录来实现。为此须使每一个文件都在文件目录中有一个目录项, 通过查找文件目录可找到该文件的目录项和它的索引结点, 进而找到文件的物理位置。对于可供多个用户共享的文件, 则可能有多个目录项。如果要将文件删除, 其目录项也应删除。

构造目录先调用 *ialloc* 为新建文件分配一磁盘 *i* 结点及内存 *i* 结点。若分配失败则返回, 分配成功时须先设置内存 *i* 结点的初值(含拷贝), 调用写目录过程 *wdir*, 将用户提供的文件名与分配给该文件的磁盘 *i* 结点号一起, 构成一新目录项, 再将它记入其父目录文件中。

3、检索目录 *namei*

用户在第一次访问某文件时, 需要使用文件的路径名, 系统按路径名去检索文件目录, 得到该文件的磁盘索引结点, 且返回给用户一个 *fd*。以后用户便可利用该 *fd* 来访问文件, 这时系统不需再去检索文件目录。

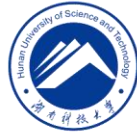
namei 根据用户给出的路径名, 从高层到低层顺序地查找各级目录, 寻找指定文件的索引结点号。检索时, 对以 '/' 开头的路径名, 须从根目录开始检索, 否则, 从当前目录开始, 并把与之对应的 *i* 结点作为工作索引结点, 然后用文件路径名中的第一分量名与根或当前目录文件中的各目录项的文件名, 逐一进行比较。由于一个目录文件可能占用多个盘块, 在检索完一个盘块中所有目录项而未找到匹配的文件分量名时, 须调用 *bmap* 和 *bread* 过程, 将下一个盘块中的所有目录项读出后, 再逐一检索。若检索完该目录文件的所有盘块而仍未找到, 才认为无此文件分量名。

检索方式采用 Hash 方法。

三、主要文件操作的处理过程

1、打开文件 *open*

检索目录。内核调用 *namei* 从根目录或从当前目录, 沿目录树查找指定的索引结点。若未找到或该文件不允许存取, 则出错处理返回 NULL, 否则转入下一步;



分配内存索引结点。如果该文件已被其它用户打开，只需对上一步中所找到的 i 结点引用计数+1，否则应为被打开文件分配一内存 i 结点，并调用磁盘读过程将磁盘 i 结点的内容拷贝到内存 i 结点中，并设置 $i.count=1$ ；

分配文件表项。为已打开的文件分配一文件表项，使表项中的 $f.inode$ 指向内存索引结点；

分配用户文件描述表项。

2、创建文件 creat

核心调用 $namei$ ，从根目录或当前目录开始，逐级向下查找指定的索引结点。此时有以下二种情况：

重写文件。 $namei$ 找到了指定 i 结点，调用 $free$ 释放原有文件的磁盘块。此时内核忽略用户指定的许可权方式和所有者，而保持原有文件的存取权限方式和文件主。最后打开。

新建。 $namei$ 未找到。调用 $ialloc$ ，为新创建的文件分配一磁盘索引结点，并将新文件名及所分配到的 i 结点编号，写入其父目录中，建立一新目录项。利用与 $open$ 相同的方式，把新文件打开。

3、关闭文件 close

根据用户文件描述符 fd ，从相应的用户文件描述符表项中，获得指向文件表项的指针 fp ，再对该文件表项中的 $f.count-1$ 。

四、主要数据结构

1、 i 节点

```
struct inode
{
    struct inode *i_forw;
    struct inode *i_back;
    char i_flag;
    unsigned int i_ino;           /*磁盘 i 节点标号*/
    unsigned int i_count;        /*引用计数*/
    unsigned short di_number;    /*关联文件数，当为 0 时，则删除该文件*/
    unsigned short di_mode;      /*存取权限*/
    unsigned short di_uid;       /*磁盘 i 节点用户 id*/
    unsigned short di_gid;       /*磁盘 i 节点组 id*/
    unsigned int di_addr[NADDR]; /*物理块号*/
}
```

2、磁盘 i 节点

```
struct dinode
{
    unsigned short di_number;    /*关联文件数*/
    unsigned short di_mode;      /*存取权限*/
    unsigned short di_uid;
    unsigned short di_gid;
    unsigned long di_size;       /*文件大小*/
    unsigned int di_addr[NADDR]; /*物理块号*/
}
```

3、目录项结构

```
struct direct
{
    char d_name[DIRSIZ];        /*目录名*/
    unsigned int d_ino;          /*目录号*/
}
```



}

4、超级块

struct filsys

```
{    unsigned short  s_isize;           /*i 节点块数*/
    unsigned long   s_fsize;          /*数据块数*/
    unsigned int    s_nfree;          /*空闲块数*/
    unsigned short  s_pfree;          /*空闲块指针*/
    unsigned int    s_free[NICFREE];  /*空闲块堆栈*/
    unsigned int    s_ninode;         /*空闲 i 节点数*/
    unsigned short  s_pinode;         /*空闲 i 节点指针*/
    unsigned int    s_inode[NICINOD]; /*空闲 i 节点数组*/
    unsigned int    s_rinode;         /*铭记 i 节点*/
    char            s_fmod;           /*超级块修改标记*/
}
```

}

5、用户密码

struct pwd

```
{    unsigned short  p_uid;
    unsigned short  p_gid;
    char            password[PWOSIZ];
};
```

};

6、目录

struct dir

```
{    struct direct  direct [DIRNUM];
    int            size;
};
```

};

7、查找内存 i 节点的 hash 表

struct hinode

```
{    struct inode  *i_forw;
};
```

};

8、系统打开表

struct file

```
{    char            f_flag;           /*文件操作标志*/
    unsigned int    f_count;          /*引用计数*/
    struct inode    *f_inode;         /*指向内存 i 节点*/
    unsigned long   f_off;            /*读/写指针*/
};
```

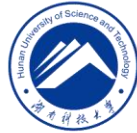
};

9、用户打开表

struct user

```
{    unsigned short u_default_mode;
    unsigned short u_uid;           /*用户标志*/
    unsigned short u_gid;           /*用户组标志*/
    unsigned short u_ofile[NOFILE]; /*用户打开表*/
};
```

};



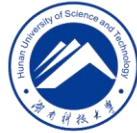
三、主要函数

- 1、i 节点内容获取函数 `iget()`
- 2、i 节点内容释放函数 `iput()`
- 3、目录创建函数 `mkdir()`
- 4、目录搜索函数 `namei()`
- 5、磁盘块分配函数 `ballocc()`
- 6、磁盘块释放函数 `bfree()`
- 7、分配 i 节点区函数 `iallocc()`
- 8、释放 i 节点区函数 `ifree()`
- 9、搜索当前目录下文件的函数 `iname()`
- 10、访问控制函数 `access()`
- 11、显示目录和文件用函数 `_dir()`
- 12、改变当前目录用函数 `chdir()`
- 13、打开文件函数 `open()`
- 14、创建文件函数 `create()`
- 15、读文件用函数 `read()`
- 16、写文件用函数 `write()`
- 17、用户登录函数 `login()`
- 18、用户退出函数 `logout()`
- 19、文件系统格式化函数 `format()`
- 20、进入文件系统函数 `install()`
- 21、关闭文件系统函数 `close()`
- 22、退出文件系统函数 `halt()`
- 23、文件删除函数 `delete()`

四、主程序说明

begin

- | | |
|--------|--|
| step1 | 对磁盘进行格式化 |
| step2 | 调用 <code>install()</code> ，进入文件系统 |
| step3 | 调用 <code>_dir()</code> ，显示当前目录 |
| step4 | 调用 <code>login()</code> ，用户注册 |
| step5 | 调用 <code>mkdir()</code> 和 <code>chdir()</code> 创建目录 |
| step6 | 调用 <code>creat()</code> ，创建文件 0 |
| step7 | 分配缓冲区 |
| step8 | 写文件 0 |
| step9 | 关闭文件 0 和释放缓冲 |
| step10 | 调用 <code>mkdir()</code> 和 <code>chdir()</code> 创建子目录 |
| step11 | 调用 <code>creat()</code> ，创建文件 1 |
| step12 | 分配缓冲区 |
| step13 | 写文件 1 |
| step14 | 关闭文件 1 和释放缓冲 |
| step15 | 调用 <code>chdir</code> 将当前目录移到上一级 |
| step16 | 调用 <code>creat()</code> ，创建文件 2 |
| step17 | 分配缓冲区 |



step18	调用 write(), 写文件 2
step19	关闭文件 2 和释放缓冲
step20	调用 delete(), 删除文件 0
step21	调用 creat(), 创建文件 3
step22	为文件 3 分配缓冲区
step23	调用 write(), 写文件 3
step24	关闭文件 3 和释放缓冲
step25	调用 open(), 打开文件 2
step26	为文件 2 分配缓冲区
step27	调用 open(), 打开文件 2
step28	释放缓冲
step29	用户退出(login)
step30	关闭(halt)

end

由上述描述过程可知, 该文件系统实际是为用户提供一个解释执行相关命令的环境。主程序中的大部分语句都被用来执行相应的命令。

下面, 我们给出每个过程的相关 C 语言程序。读者也可以使用这些子过程, 编写出一个用 shell 控制的文件系统界面。

五、参考程序

具体参见 <http://www.doc88.com/p-918959409953.html>

C 组（挑战题）

C 类实验的思路来自《南京大学计算机系统基础课程（Introduction to Computer System, ICS）》中的编程作业，一次作业称为一个 PA（Programming Assignment），这一系列作业的最终成果是 NEMU（南京大学模拟器）。其中 PA0 主要关注开发环境的配置，为学生搭建后续实验的基础平台；PA1 与 PA2 重点在于实现一套指令集，从而理解计算机指令执行的基础环境（AM, Abstract Machine）。PA3 的核心目标是实现一个简化的批处理操作系统，涉及进程管理、内存分配和 I/O 子系统等操作系统基本功能。PA4 与 PA5 进一步探索虚拟化、上下文切换以及优化，最终形成一个具备虚存管理的分时多任务操作系统（Nanos-lite，南京大学操作系统裁剪版）。

本课程组设计了简化版的 PA 系列作业，最终成果是 OS256，OS256 与 NEMU 的主要区别在于：

- 1、用汇编代替 C 语言，由学生自己写汇编器。
- 2、用自定义的超小规模指令集代替了 X86 指令集。
- 3、用 WEB 平台代替 LINUX 平台。
- 4、大大缩小指令集规模和代码规模。
- 5、提供可视化工具。
- 6、提出了棋盘推演这一教学方法和学习方法。
- 7、提出了“指令合并”这一降低理解难度的方法。

C 类实验的主要内容包括，学生从零开始自行搭建一个操作系统可视化平台 OS256，其中包含一个硬件模拟器 PC256、一个汇编器 assembler256，并在其中开展操作系统相关实验，最终在 OS256 中实现一个微型操作系统。由以上描述可见，C 类实验是闯关式实验，学生必须按顺序完成前驱实验，才能理解和完成后继实验，因此指导教师可根据学生最终到达的实验楼层来判断学生对操作系统知识体系的掌握程度。

OS256 已获颁软件著作权，证书登记号 2024SR0434345，严禁直接以 OS256 的代码作为论文主要内容。

与 OS256 有关的问题，可以加 QQ 群 1031560262 讨论。

OS256 相关教学视频，请在 B 站搜索“OS256”。

题目 C1：熟悉 OS256

1、实验目的

(1)理解计算机控制流的基本原理。通过搭建一个极简的硬件模拟器 PC256，理解 IP 如何驱动指令的依次执行，掌握控制流的本质——即 IP 寄存器中存储过的值所组成的序列。

(2)掌握指令集与寻址模式的设计方法。理解操作码（Opcode）与操作数（Operand）的编码方式，掌握立即寻址、直接寻址、寄存器寻址、寄存器间接寻址四种寻址模式的原理与实现。

(3)理解汇编器的工作原理。通过实现一个完整的汇编器（Assembler256），掌握从汇编语言到机器码的翻译过程，理解“四分五列”的汇编格式、标号解析、伪指令（DB、UNTIL）等核心概念。

(4)建立从硬件到软件的完整认知链条。通过“机器码→汇编→操作系统雏形”的递进式实验，理解操作系统作为软件是如何在硬件上运行的。

2、设计思路

本实验采用“最小化模拟”的设计理念。PC256 模拟器仅模拟硬件对外的状态变化（寄存器与内存中存储的值），不考虑内部电信号的传播、门电路、MOS 管等底层物理细节，从而将复杂度控制在适合本科教学的范围内。

整体架构分为三层：

硬件模拟层（hardware.js）：模拟 CPU 中的寄存器（IP、A、B、C、D、Z 等）和 256 个内存单元（RAM[0]~RAM[255]），实现指令的取指、执行周期。

汇编层（assembler.js + opcodes.js）：将人类可读的汇编助记符（HLT、NOP、JMP、MOV、ADD 等）翻译为机器码，并支持标号、伪指令等高级特性。

用户界面层（index.html+OS256.js+float.js）：基于 Vue.js 实现双向绑定，提供代码输入、单步执行、连续执行、快照回放、源码高亮等交互功能。

核心教学方法是“棋盘推演”——将内存视为棋盘，IP 寄存器视为棋子，指令执行就是棋子按照规则在棋盘上移动的过程。这种可视化方法使抽象的控制流概念变得直观、所见即所得。

3、设计内容

文件夹“C 类实验 1 熟悉 OS256”包含 5 份 PPT 和 1 个子文件夹，以及第三方框架 vue.global.js。

请你按照《0.0 从零搭建 OS256.pptx》的描述一步一步实现操作系统的实验平台——OS256，共分为 31 步。请至少学习到“第 22 步实现 CMP、JE、JNE”。如果遇到问题，可以参考子文件夹中从第 3 步到第 31 步的工程快照，最后的快照“第 31 步时间机器”是 C 类实验 1 的最终成果，也是 C 类实验 2 的起点。另有 4 份 PPT 用于描述 OS256 的设计思路和

使用方法，供参考。

在此基础上，请参考 JE 和 JNE 的代码，实现两个新指令 JA 和 JB：

指令助记符	全称	含义
JA	Jump if Above	大于则跳转
JB	Jump if Below	小于则跳转

要求和提示：

- ①必须支持 4 种寻址方式。
 - ②观察 opcodes.js 文件中的数字以找到合适的连续机器码。
 - ③修改 CMP 的代码，用 Z=1 表达相等，Z=2 表示大于，Z=3 表示小于。
 - ④修改 UI，使得 Z 寄存器的名字根据其值而变化，即“Z 等于”“Z 大于”“Z 小于”。
- 请阐述设计思路、源码实现、实验验证、心得体会。

题目 C2：单道批处理

1、实验目的

(1)理解操作系统的本质角色。通过对比“无 OS 的串行执行”与“单道批处理 OS”的差异，理解操作系统作为“管家”的核心职责——管理硬件资源、为应用程序提供服务、保障系统安全与公平。

(2)掌握段机制与权机制的原理与实现。理解如何通过段寄存器限制应用程序的访问范围，如何通过 CPL（Current Privilege Level）区分特权指令与非特权指令，以及二者如何协同工作以构建系统的安全边界。

(3)理解系统调用、异常与中断三种控制流转移方式的异同。掌握基于 IDT（Interrupt Descriptor Table）的跳转表机制，理解系统调用（主动请求服务）、异常（被动响应错误）和中断（外部事件触发）在触发条件、参数传递和处理流程上的区别与联系。

(4)理解中断处理的完整流程与 IF 开关的作用。掌握时钟与闹钟硬件的工作原理，理解开中断（STI）与关中断（CLI）对 CPU 指令执行阶段的影响，理解“中断驱动”是现代操作系统的核心驱动力。

2、设计思路

本实验基于上一阶段搭建的 OS256 硬件模拟器与汇编器，以“管家与客人”的隐喻贯穿始终：管家（OS）是机器的管理者，负责招待客人（APP）、管理房间（内存段）、维护秩序（特权检查）。客人（APP）是被服务的对象，可能是守规矩的好客人，也可能是“蠢”或“坏”的客人。

本实验设计了五个攻防场景，由客人进行攻击，由管家负责防御。攻击方式有 5 种，防御机制也有 5 种，这就是 5 大机制。每个新机制的产生，都是有原因的，都是为了封堵前一个阶段暴露的安全漏洞，在不断的“攻”与“防”过程中，可以深刻理解操作系统的设计哲学。

机制	解决的问题	核心指令/寄存器
段机制	APP 篡改 OS 代码	segment_start/end setseg/jmpAPP
权机制	APP 执行特权指令	cpl, jmpAPP 加入特权检查
系统调用	APP 无法安全返回 OS	jmpOS, IDT, setIDTR
异常机制	APP 执行非法操作导致系统崩溃	扩展 div 指令, IDT 异常入口
中断机制	APP 死循环导致 OS 无法夺回控制权	setAlarm1, clock, INTR, IF

为什么在设计一个操作系统时，要把安全问题放在首位？必须深刻理解“管家与客人”的隐喻——这不是安全问题，是生存问题。操作系统与企业电力系统、汽车传动系统、城市交通系统的本质区别在哪里？城市交通系统中，也有代码要运行，这些代码也是管家，但城市交通系统中**没有客人**；此时管家为主人也就是人类服务，但没有客人需要服侍；城市交通系统不需要从网上下载 APP，不区分内核和应用程序，不需要设计段和权这种机制。操作系统天生是为**外来代码**服务的本土代码，外来代码是入侵者，因此本土代码必须有控制外来代码的能力。这种能力不可能仅仅来自于软件（因为外来代码也是软件，也有控制 CPU 来执

行的机会)，而必然是“软硬结合”的机制。比如，段机制依赖于段寄存器，权机制依赖于 CPL 寄存器，系统调用、异常和中断都依赖于 IDT。

3、设计内容

文件夹“C 类实验 2 单道批处理”包含 7 份 PPT、1 个子文件夹和 1 个压缩文件。

子文件夹中保存的是，在上一题最终成果“第 31 步时间机器”基础上，提前加入 `setseg`、`jmpAPP`、`jmpOS`、`setIDTR`、`setClock`、`setAlarm1`、`setAlarm2` 指令并修改 `DIV` 指令的快照“第 32 步用于实现五大机制”，这么做的目的是减少学习的难度，让学习者只需要注销和反注销已有代码，而不需要四处粘贴代码。7 份 PPT 用于阐述单道批处理操作系统涉及的五大机制——段、权、系统调用、异常、中断，其中包含的代码都在“第 32 步用于实现五大机制”中运行，所有对硬件的修改都针对该快照。

请你按顺序学习 7 份 PPT，在工程快照的基础上逐步引入新的机制，理解和掌握五大机制的作用和机理。最后请按照压缩文件中的要求进行实验并回答问题：

Q1:在代码中 X 和 Y 各自的段和权是多少？

Q2:X 和 Y 段的范围和权的高低是在哪一行代码被决定的？

Q3:OS 与 X 和 Y 都是软件，身份相同，凭什么 OS 可以决定 X 和 Y 的段和权，X 和 Y 却不能决定 OS 的段和权呢？

Q4:当段机制不存在时，权机制可以被 APP 轻易取消。为了说明这一点，请先将 `jmpAPP x_start,x_start,255` 修改为 `jmpAPP x_start,0,255`，这意味着对于 X 而言，段机制被取消。然后请你出手，在仅修改 X 的代码的前提下，让 X 能在高特权等级下运行自己的代码。

Q5:当权机制不存在时，段机制可以被 APP 轻易取消。为了说明这一点，将 `jmpAPP x_start,x_start,255` 修改为 `jmp x_start`，同时在 X 代码的最前面增加一句 `setseg X_start,255`，这意味着对于 X 而言，权机制被取消。在仅修改 X（`setseg X_start,255` 这一行之后）代码的前提下，请让 X 的段变为 `[0,255]`，并篡改 OS 的某处代码作为证明。

Q6:即使段机制和权机制都存在，目前 OS256 仍有安全漏洞。比如说，`setIDTR` 目前只是一个非特权指令，请利用这个漏洞，在仅修改 X 的代码的前提下，让 X 能在高特权等级下运行自己的代码，且段范围为 `[0,255]`。

题目 C3：多道批处理与进程

1、实验目的

(1)理解单道批处理操作系统的固有缺陷。通过对比单道与多道的运行效率，深刻理解 CPU 与 I/O 设备之间、不同 I/O 设备之间因无法并行而导致的资源浪费问题。

(2)掌握进程的核心概念。理解进程作为“正在执行的程序”的本质，掌握进程控制块（PCB）的结构与作用，理解进程状态（就绪、运行、阻塞、退出）及其转换条件。

(3)理解上下文切换的完整机制。掌握 save（保存现场）与 load（恢复现场）的时机与内容，理解中断、异常、系统调用三种控制流转移方式在进程管理中的统一处理框架。

(4)理解进程调度的基本原理。掌握先来先服务（FCFS）调度算法的实现，理解就绪队列与阻塞队列在进程管理中的作用，了解时间片轮转调度的基本思想。

2、设计思路

本实验在“单道批处理操作系统”的基础上，以“中断地狱”问题为驱动，逐步引入进程管理的核心机制：

核心问题：在单道批处理 OS 中引入多道后，一个 APP 可能被另一个 APP 发出的中断信号“击中”，导致控制流永久丢失，程序无法完整执行。

解决方案：借鉴游戏“存档/读档”机制，设计 save/load 方案：

当进程被打断时（系统调用、异常、中断），由硬件自动执行 save，将 CPU 中所有用户可写寄存器的值保存到进程的 PCB 中；

当 OS 决定恢复该进程时，执行 load，从 PCB 中恢复所有寄存器的值，使进程从“从被打断”的地方继续执行。

核心数据结构——PCB（进程控制块）：

偏移	字段	说明
+0	PID	进程标识号
+1~+4	A~D	通用寄存器
+5	IP	程序计数器（恢复点）
+6	Z	零标志寄存器
+7	Status	进程状态（R=就绪，B=阻塞，E=退出）
+8	COUNT	调度计数/时间片
+9	MM_pointer	指向内存管理结构
+10~+11	段首/段尾	段机制的段边界
+12	CR3	页表寄存器

双队列调度模型：

就绪队列：存放所有可被调度执行的进程

阻塞队列：存放等待 I/O 完成的进程（每个设备一个阻塞队列）

调度器从就绪队列取出队首进程运行；I/O 完成后，进程从阻塞队列移回就绪队列

3、设计内容

文件夹“C 类实验 3 多道批处理之进程”包含 3 份 PPT、1 个子文件夹和 1 个压缩文件。

3 份 PPT 用于阐述多道批处理操作系统的优势、特点和实现，特别强调进程的设计与实现，进程创建的具体过程。子文件夹保存了上一题的结果，即工程快照“第 33 步已实现五大机制”。

请你按顺序学习 3 份 PPT，在工程快照的基础上逐步实现进程的描述和控制，实现进程创建系统调用。最后请按照压缩文件中的要求进行实验并回答问题：

代码理解题：对 OS256 代码的理解。

教材分析题：将教材上的文字，与代码一一对应。

两种题型各自有 33 道题目。

对于每种题型，请至少回答 10 道题。

题目 C4：互斥同步

1、实验目的

(1)理解并发的本质与并发源。掌握并发(Concurrency)的两种形式——交替(Interleave)与重叠(Overlap)，理解单处理器上由于调度导致的并发问题，以及多处理器上指令真正并行的场景。

(2)掌握互斥(Mutual Exclusion)问题的成因与解决方案。理解共享资源竞争导致的数据不一致问题，掌握信号量(Semaphore)解决互斥问题的核心范式——上P下V，两头堵。

(3)掌握同步(Synchronization)问题的成因与解决方案。理解前驱-后继关系对控制流执行顺序的要求，掌握信号量解决同步问题的核心范式——前V后P，来接力。

(4)理解生产者-消费者模型及其在同步与互斥中的综合应用。掌握多生产者、多消费者场景下，如何通过多个信号量分别实现同步与互斥，理解“固差”与“固模”两种同步约束的数学本质。

2、设计思路

本实验承接“多道批处理操作系统”的实验基础，以“共享带来的问题”为切入点，逐步展开并发编程的核心机制：

核心问题：在进程/线程共享资源的环境中，不同控制流的执行顺序不可预测，导致两类典型问题：①互斥失败：两个进程同时访问共享资源（如共享变量），后执行的写操作覆盖前者的结果；②同步失败：本应后执行的任务（如播种）抢在了前驱任务（如锄地）之前执行。

为了解决方案以上问题，OS256用汇编语言实现了信号量，而非仅提供抽象的应用编程接口，这非常有利于初学者理解信号量的功能，理解信号量是如何解决互斥与同步问题的。

关键设计决策：信号量的P和V必须实现为系统调用，以确保其原子性——在单处理器上，系统调用执行期间关中断，避免了调度导致的并发干扰。

3、设计内容

文件夹“C类实验4互斥与同步”包含3份PPT、1个子文件夹。

其中2份PPT用于阐述并发、互斥、同步的基本概念，以及OS256实现的信号量机制。子文件夹“第34步已实现save和load”用于实验和作业。

请你按照《编程实践小作业4互斥同步》中的要求进行实验并回答问题：

Q1:运行本代码，为什么输出的字符串是AAAAA...?

Q2:反注销setclock 11，为什么输出的前4个字符为AA□A?

Q3:请使用信号量解决这个问题，令其不输出□，此时的输出是什么?

Q4:在3的基础上，请在合适位置增加PV操作，使其最终输出BABABABA...

Q5:在3的基础上，重新调整PV操作，使其最终输出BBABBABBABBA...

题目 C5：死锁

1、实验目的

(1)理解死锁的本质与成因。掌握死锁的四个必要条件（互斥、占有且等待、不可抢占、循环等待），理解死锁是“一组相互竞争资源的进程间的永久阻塞”。

(2)理解内部排序与外部排序对死锁的影响。掌握进程内指令顺序（内部排序）与进程间调度顺序（外部排序）如何共同决定系统是否进入死锁状态，理解“模式 1（可能死锁）”与“模式 2（一定不死锁）”的本质区别。

(3)掌握死锁预防的基本方法。理解如何通过改变内部排序（如“左右撇子”策略）或限制资源分配（如“四人上桌”策略）来预防死锁，并在 OS256 中验证这些方案的有效性。

(4)理解死锁避免的核心思想。了解银行家算法的工作原理，理解“安全状态”与“不安全状态”的区别，以及为什么“不安全状态”不一定导致死锁。

2、设计思路

本实验以“弓箭双兵”和“哲学家就餐”两个经典案例为线索，逐步揭示死锁问题的本质：

核心分析框架——两个排序决定死锁：

概念	定义	谁决定	控制难度
内部排序	一个进程内部获取/释放资源的顺序	程序员（代码顺序）	可控
外部排序	多个进程在 CPU 上执行的相对顺序	调度器（时间片/中断）	不可控

实验基于 OS256 教学平台，以“弓箭双兵→哲学家就餐”的递进式案例，引导实验者从最小规模到大规模理解死锁问题的本质。通过程序 5-3 和 5-4 的动手实验，实验者将直观感受死锁的发生与预防，为后续学习银行家算法等死锁避免技术奠定基础。

3、设计内容

文件夹“C 类实验 5 死锁”包含 2 份 PPT、1 个子文件夹。

1 份 PPT 用于阐述死锁的基本概念和解决方案，其中的代码，请使用子文件夹中的 OS256 来运行。请你按照《编程实践小作业 5 死锁》中的要求进行实验并回答以下 8 个问题：

程序 5-3、小规模死锁案例：弓箭双兵

请参照情况 1 和情况 2，讨论情况 4~8

情况 4，修改进程 X 的 4 个操作为 AABB 进程 Y 为 BABA，(图 6.3)，有没有死锁？

情况 5，修改进程 X 的 4 个操作为 ABBA 进程 Y 为 BABA，运行后有没有死锁？

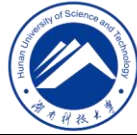
情况 6，修改进程 X 的 4 个操作为 ABAB 进程 Y 为 BAAB，运行后有没有死锁？

情况 7，修改进程 X 的 4 个操作为 AABB 进程 Y 为 BAAB，运行后有没有死锁？

情况 8，修改进程 X 的 4 个操作为 ABBA 进程 Y 为 BAAB，运行后有没有死锁？

程序 5-4、大规模死锁案例：哲学家就餐问题：5 个进程代表 5 个哲学家

;1、运行本代码，为什么输出的字符串是 11111，然后死锁？



湖南科技大学
Hunan University of Science and Technology

-
- ;2、请实现教材 P177 图 6.13 的方法，避免死锁
 - ;3、请实现教材 P193 题 6.18 的方法，让任一哲学家成为右撇子，避免死锁

题目 C6：调度

1、实验目的

(1)理解调度的本质与三要素框架。掌握短程调度（Short-term Scheduling）的核心概念，理解一次调度决策由“调度时机”“调度流派”“选择函数”三个要素共同决定，能够用三要素框架完整地描述任意调度算法。

(2)理解时钟中断在调度中的核心作用。掌握时钟中断作为操作系统“心跳”的机制原理，理解其在时间统计、抢占式调度、进程时间片管理中的关键地位，认识“没有时钟中断就没有现代抢占式调度”这一重要结论。

(3)掌握六种经典调度算法的原理与实现。深入理解 FCFS、RR、SPN、SRT、HRRN、MLFQ 六种调度算法的三要素构成与选择函数，能够分析不同算法对进程调度顺序的影响，理解其各自的优缺点与适用场景。

(4)理解调度算法的工程实现路径。掌握基于 PCB “计分板”的调度实现方法（统计 w 、 e 、 s 、 a 等时间参数）与基于队列的工程实现方法，理解两种路径的差异与各自的适用场景。

2、设计思路

本实验以“调度三要素”为分析框架，将调度算法的描述从“模糊的自然语言”提升为“精确的工程语言”：

调度三要素：

要素	含义	代码体现
调度时机	何时触发调度决策	jmp scheduler 的位置
调度流派	抢占还是非抢占	jmpAPP_PCB [current] vs jmp scheduler
选择函数	依据什么标准选进程	schedule:标号后的比较逻辑

调度时机：利用前述章节的代码进行实践验证。

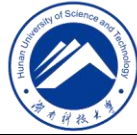
调度流派：抢占与非抢占，设计了“程序 6-1”进行实践验证。

选择函数：从“计数板”到“队列”，设计了“程序 6-2”和“程序 6-3”进行实践验证。

“程序 6-2”和“程序 6-3”是两套调度实现方案，可以从不同角度理解调度的本质：

(1)基于 PCB 计分板的实现（教材原理版）：为每个进程维护 w （等待时间）、 e （执行时间）、 s （服务时间）、 a （到达时间）等统计字段，选择函数通过计算这些值来决策。这种方式贴近教材理论描述，便于理解调度算法的数学本质。

(2)基于队列的工程实现（工程实践版）：利用队列的 FIFO 特性天然表达“等待时间”，利用 PCB 中的 counter 字段表达时间片，无需显式计算 $w/e/s/a$ 。这种方式贴近工程实践，效率高、代码简洁。



3、设计内容

文件夹“C 类实验 6 调度”包含 3 份 PPT、1 个子文件夹。

2 份 PPT 用于阐述调度的三要素理论并讨论了 6 种调度算法。请你按照《编程实践小作业 6 调度.pptx》中的要求进行实验并回答以下 5 个问题：

“程序 6-1”，请根据所学的理论知识，说明为什么在激活 jmpAPP_PCB [current]之后，轮转算法就变成了 FCFS 算法。

“程序 6-1”，利用 count，将当前 $q=1$ 的轮转变化为 $q=4$ 的轮转。（提示：修改 setclock 0 为 setclock 3 是错误解法）

“程序 6-2”，请回答，在 2200 回合，RAM 中右上有两列“3 6 4 5 2”，它们表示什么？

“程序 6-3”，请模仿程序 6-1，将其变为 FCFS 算法和 $q=4$ 的轮转。

“程序 6-3”，修改本代码能实现教材中的“最短进程优先”和“最短剩余时间”“最高响应比优先”么？请陈述你的理由

题目 C7：分段分页

1、实验目的

(1)理解“码中藏址”与重定位的必要性。通过对比“本地机器码”与“远方机器码”的执行差异，理解编译时生成的逻辑地址（LocationCounter）如何嵌入指令，以及重定位（Relocation）对于运行独立编译程序的关键作用。

(2)掌握分段式内存管理的基本原理与实现。学习在 readRam/writeRam 中增加段首偏移，实现逻辑地址到物理地址的动态转换；掌握段首/段尾寄存器的保护作用，并能修改 PCB 中的段信息使多个“远方”程序同时正确运行。

(3)掌握分页式内存管理的核心机制。理解页、框、页表的概念，学会用“一刀两段，查表换头”的速算法完成地址翻译；实现一级页表、二级页表，并对比其内存开销。

(4)理解页表共享与地址空间布局。通过实现“OS 借用进程页表”的设计，理解用户态与内核态共享地址空间的高效性，并体会现代操作系统（Windows/Linux）中用户区与系统区划分的思想。

(5)培养调试与优化能力。在有限的 256 字节内存中，通过调整页表、修改代码布局，解决外部碎片和内存不足问题，体验实际 OS 内存管理的权衡。

2、设计思路

本实验在 OS256 模拟器上，以“程序 3-3”（多道进程管理）为基准，逐步引入分段和分页机制。所有实验均基于已开启分段/分页的修改版 OS256（教师提供或按 PPT 自行修改）。

从“不能运行远方代码”出发：先运行未启用重定位的版本，观察因逻辑地址冲突导致的错误，引出分段方案——在 readRam/writeRam 中自动加上段首偏移。

分段实验：学生需修改进程的 PCB 中段首/段尾，以及 org 伪指令，使多个 APP 在各自段内正确运行。同时完成 P0 进程的代码重定位练习。

分页实验：从“映射表版”开始，理解离散存储；再过渡到“页表除法/移位版”，掌握高效地址翻译。随后完成“瘦身”任务——压缩页表、合并代码页、引入二级页表，解决内存紧张问题。

高级设计：注销 OS 独立页表，让 OS 共享进程页表，观察无需切换 CR3 的系统调用优化，并分析这种设计对性能的积极影响。

3、设计内容

文件夹“C 类实验 7 分段分页”包含 2 份 PPT、1 个压缩文件。

2 份 PPT 从理论上阐述了内存管理的分段方案、分页方案，压缩文件中则是实验代码和实验指导 PPT。

请按照压缩文件中实验指导 PPT 的要求进行实验并回答问题。

附录：Linux 编程基础

1、实验目的

- (1) 熟悉 Linux 下的 C/C++编程环境：vi/gedit 编辑器 + gcc 编译器 +gdb 调试器。
- (2) 学会使用 VMware 虚拟机软件安装 Linux 操作系统。

2、实验内容和步骤

1) 使用 VMware 虚拟机软件安装 Linux 操作系统

步骤 1: VMware workstation 12 Pro 的下载和安装。安装方法很简单，直接双击安装程序，默认安装即可。安装完成后，点击桌面的快捷启动图标，会提示你输入许可证密钥，输入密钥激活软件。

步骤 2: 利用 VMware 创建一台虚拟机，根据自己的电脑具体情况，设置好虚拟机硬件配置，一般默认配置就好，更改光驱设置，使用 ISO 映像文件来安装 linux 系统。

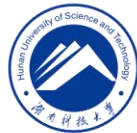
步骤 3: 使用 VMware 虚拟机安装 Linux 操作系统。

2) 熟悉 Linux 下的 C/C++编程环境：vi/gedit 编辑器 + gcc 编译器 +gdb 调试器

要求学生熟悉 Linux 系统，熟练使用 vi 编辑器或 gedit 编辑器，掌握如何在 Linux 下编写并编译、调试、运行程序。使用 c 语言编写一个显示输入文件内容的程序，要求显示时删除输入文件中多余的空行。

- (1) 输入c程序可以用 vi/gedit 命令来完成。下面就使用vi命令/gedit来输入以下c程序cowpact.c。vi 或 gedit 的使用请上网查询相关的指南。

```
/*Remove newlines */
#include <stdio.h>
main()
{
    int c, n = 0, max = 1;
    while ((c = getchar()) != EOF)
    {
        if (c == '\n')
            n++;
        else
            n = 0;
        if (n <= max)
            putchar(c);
    }
}
```



}

上述程序是用来删除空行的，连续两个以上的空行只保留一个空行。

(2) 编译 c 程序

由于 c 语言是一种高级语言，所以输入完 c 程序后就要对它进行编译。gcc 命令可以用来编译 c 程序。如果在 gcc 命令后面直接跟上文件名，则编译后的输出结果将存放在标准的 a.out 文件中。如果 gcc 命令使用 -o 选项，则可以将编译结果存放在自己命名的文件中。为方便起见，我们使用带 -o 任选项的 gcc 命令来进行编译。当然，在本次课程设计中也可以使用 gcc 命令进行编译。

```
$gcc -o compact compact.c
```

这样，编译结果就存放在 compact 文件中。如果出现编译错误，则需要使用 gdb（参见参考资料[2]）进行调试并找出错误，然后再利用 vi 命令来对程序进行修改。

(3) 执行 c 程序

由于存放编译结果的文件本身就是可执行文件，所以可以在 shell 提示符敲入该文件的名称来执行它，在有些情况下还要提供输入内容。下面给出 compact 的执行过程。

首先创建测试文件 testfile。

```
$cat > testfile
```

```
this is line 1
```

```
this is line 2
```

```
this is line 4
```

<ctrl+d> （这里表示的是在键盘上同时按下 ctrl 键和 d，终止输入）

```
$
```

然后，对 testfile 文件使用 compact。

```
$./compact < testfile
```

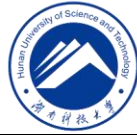
```
this is line 1
```

```
this is line 2
```

```
this is line 4
```

```
$
```

这样就从输入文件中删除了多余的空行



3、参考资料

本实验的内容是要通过编写和调试一个模拟系统动态分配资源的银行家算法程序,有效地避免死锁发生。具体要求如下:

[1]Linux 操作系统下 C 语言编程快速入门, <http://www.doc88.com/p-43941787254.html>

[2]程序调试工具 GDB 用法, www.cnblogs.com/itech/archive/2011/02/12/1952888.html